श्री ज्ञानेश्वराय नमः

# Responsibilities of Data Link Layer:

- flow control → S&W, GBN, SR  DLL uses go back N.
- error control → CRC, checksum  DLL uses CRC.
- framing
- physical addressing
- access control

---

## Framing:

we'll put data into frames. A frame would have start and end delimiters.

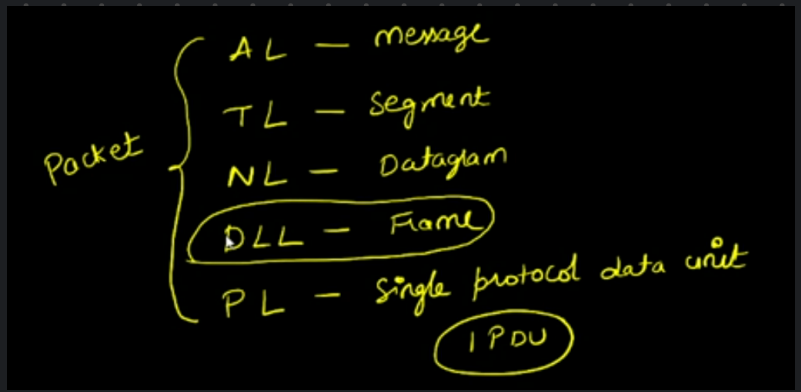



Frames

- Fixed length
  - → no need to specify length
- Variable length
  - → specify length or end delimiter



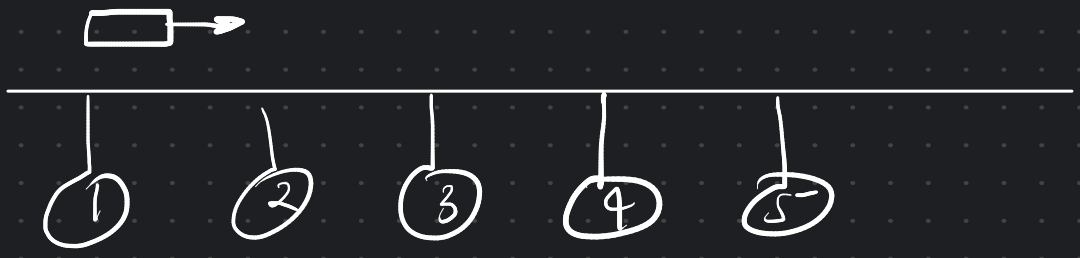

A packet is called diff. names in diff. layers.



If frame has variable size, we'll add an end delimiter to know the packet end.
Say something like '$'.
There'll be a start delimiter in the header.

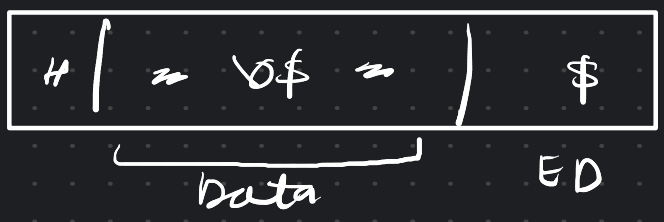



whenever many nodes are connected and data is being transmitted, all stations should know where the data starts and ends.

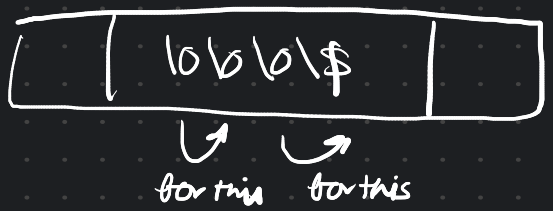



In the 1980's, only text data was transmitted over the internet. For the end delimiter, a special character (like $) was used. Now other stuff apart from text is also sent. And the character used for the E.D. could be present in the data. That point in the data could be mistaken as the end of the data. So instead, the escape character is used ('\0') before the special character.

It is called character stuffing.




What if '\0$' wants to be a part of the data?
Use 2 escape characters: one for the first escape character and one for the $




This data would actually mean \0 $

Bit Stuffing:

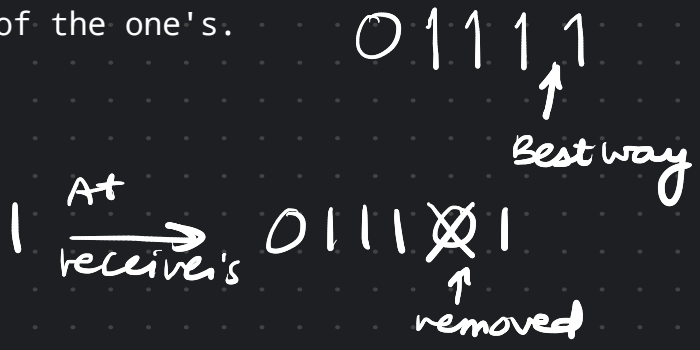



Say the end delimiter is 01111

What if the data has 01111?
the receiver would wrongly think of it as the end of the frame.

So we break the sequence.

01111 -> we insert a zero between any of the one's.




If data contains 01111, then we send 011101.
At the receiver's, the extra zero is removed.

If the data is 011101 in the first place, the receiver will remove the extra zero. So at the receiver's, if we get 01110, the 0 will be removed if the bit next is either 0 or 1.

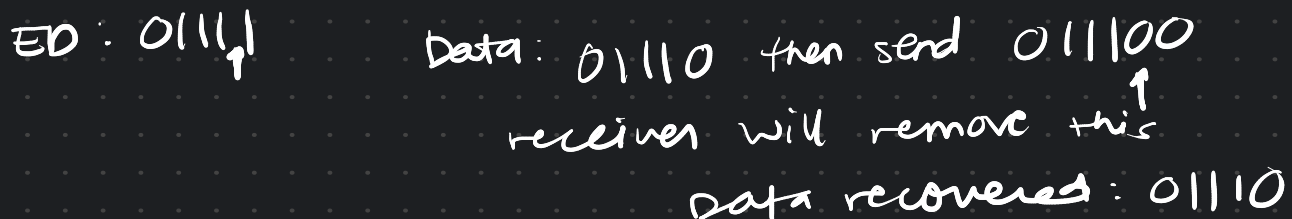
We add a 0 even if the next bit is 0/1. Accd. to our pattern, we add extra zero.

If the ED is 01111 and we initially decide to remove the extra 0 after 3 1's at the receiver's, then we add 0 after every occurence of '0111' even if it isn't a 1 bit ahead of it.

say data is 011100
we'd still send 011100

at receiver's, the extra zero would be removed.

the original data 01110 would be recovered.



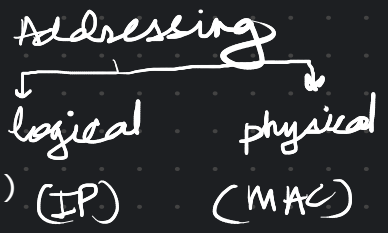

Q. ED = 011111, Data = 01010111101111100. What is transmitted?

→ 0101011110111111100

0 0

ED is 0 followed by five 1's. Add 0 after every occurence of 0111 (0 followed by four 1's).

# Physical Addressing: Data Link Layer

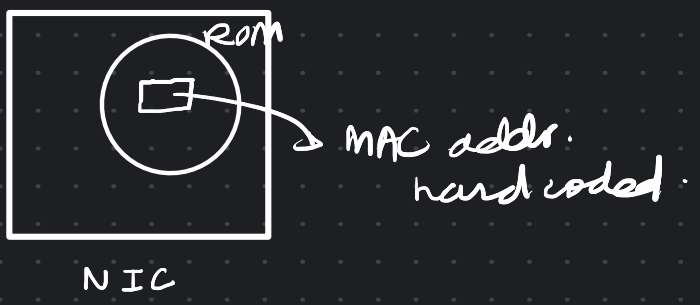



LA: any address that is unique in the internet
PA: any address that is unique in a network (LAN, subnet, etc.)

IP address is a 32-bit #. It is a s/w number, not permanent.
An IP addr. has 2 parts: NID and HID.
NID part is useful for routing.




The MAC address is a 48-bit #. It is a h/w number, permanent.
It is present inside the NIC (Network Interface Card).




A MAC address has 3 parts:
vendor ID, date, serial #

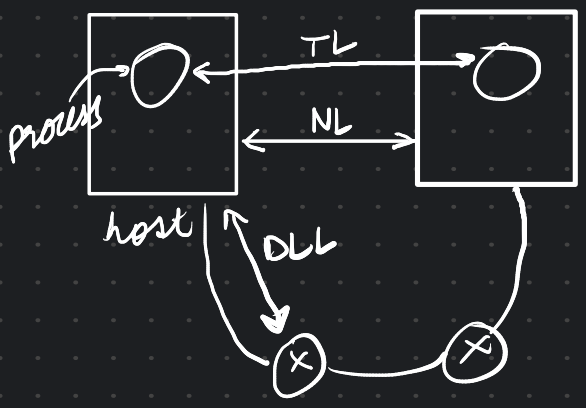



We can use an IP address as a logical address.
We cannot use a MAC address as a logical address,
because it is not useful for routing.

Data link layer is divided into:
- LLC (logical link control): takes care of flow control, error control
- MAC (media access control): framing, access control, addressing, error control

Network Layer: 3rd layer

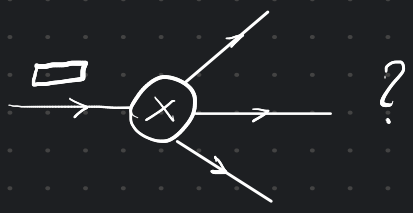
- host-to-host connectivity




DLL: hop to hop
NL: host to host
TL: end to end
or process to process

- logical addressing

- switching: which link to send packet to?




- routing: creating routing table

- congestion control:
  IP doesn't do congestion control

- Fragmentation:
  on a network, there'd be a maximum transmittable unit (MTU).
  So packets must be fragmented not to exceed the MTU.

Congestion: Traffic becomes heavy and packets are dropped.

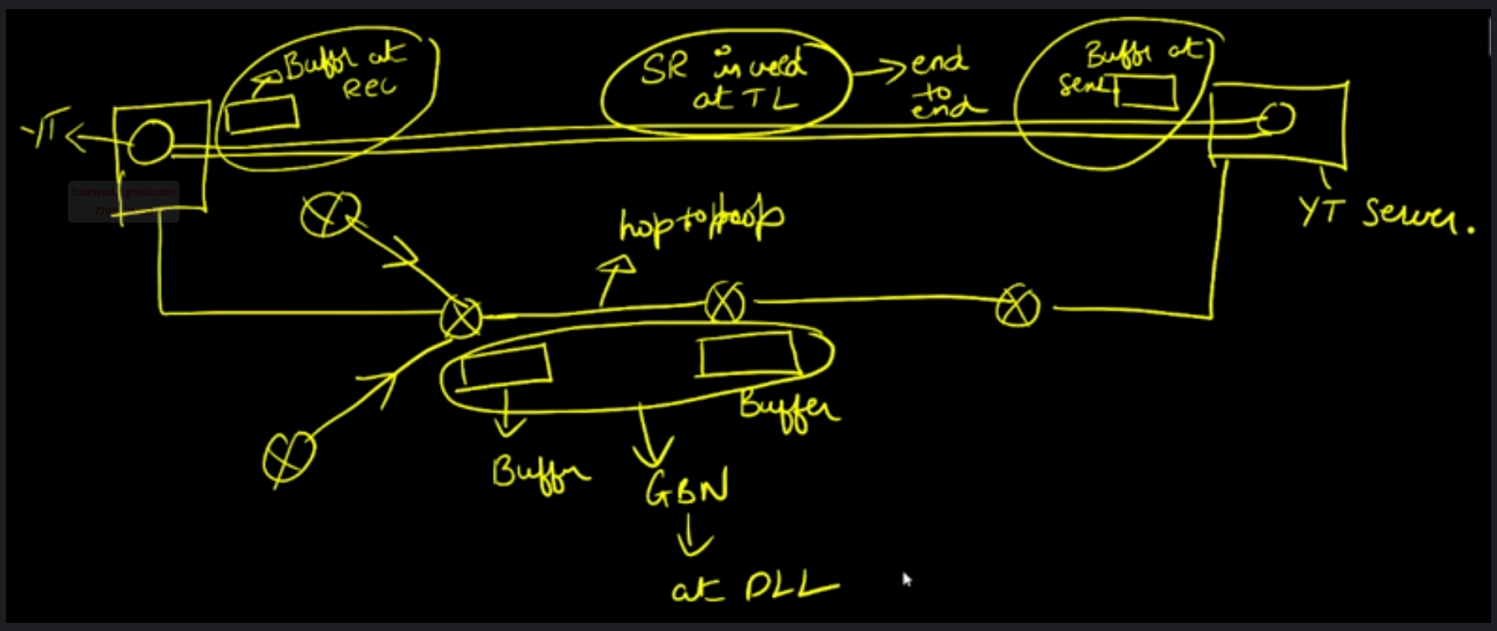
Routing means creating routing tables, switching means using them.

Transport Layer: 4th layer

- end-to-end connectivity
- flow control { stop and wait, go back N, SR } (TL uses SR)
- error control { crc, checksum } (TL uses checksum)
- segmentation:

  Application layer will send huge messages to TL.
  The TL will break them into segments and
  forward them to the layers below.

- multiplexing and demultiplexing
- congestion control (both the resp of NL and TL)




---

TL → end-to-end → TCP

NL → hop to hop → IP

why do you need TCP if IP can deliver packets?

Think of it as such. You want to send a letter and some stuff to your crush.
You send a courier. Say she's gone out when the stuff reaches her home.
Her father would see it and throw all the stuff away. Also the stuff you send
might get damaged due to the way it is unpacked.
So before you send the courier, you call her and tell her to open the items
in a particular order.

IP is like the postal service. Best effort, no guarantee. Connectionless (receiver not ready, packets might get discarded.)

TCP is like the telephone service. Connection Oriented. (asking to be ready with buffers)
Reliable. It'll ask for retransmission.

Session Layer:

- authorization (resource access) and authentication (credentials)

- checkpoints:
  like in torrents. When you download huge files, progress is saved at certain points. Say the connection is lost, the download is resumed from the checkpoint instead of the beginning.

- Synchronization:
  think of it as combining video and audio

- dialog control:
  say there's a web conference and multiple people are connected. At any time, only 1 person can speak.

- logical grouping:
  grouping the operations together

  atomicity: like in ACID properties

Presentation Layer

- character translation: ASCII, Unicode

- Encryption/ Decryption: to maintain privacy

- Compression: used to reduce size

Application Layer:

depends on the application, not related to networking

Ethernet -> IEEE 802.3

- topology: bus

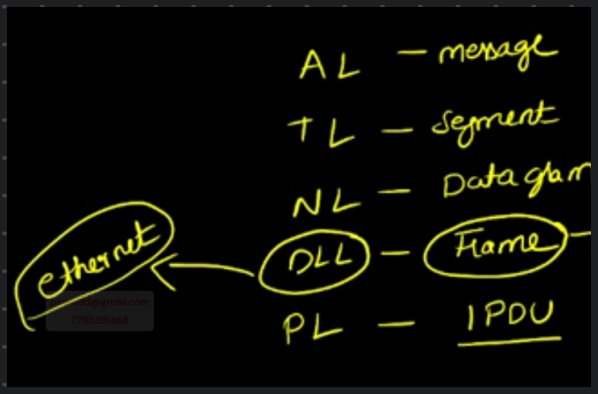
- access control method: CSMA/CD

- no ACKs are used

- bandwidth: 10 Mbps or 100 Mbps or 1Gbps

  - standard ethernet
  - fast ethernet
  - Gigabit ethernet

- ethernet deals with the DLL
  (a packet is called a frame)




Frame format of ethernet:



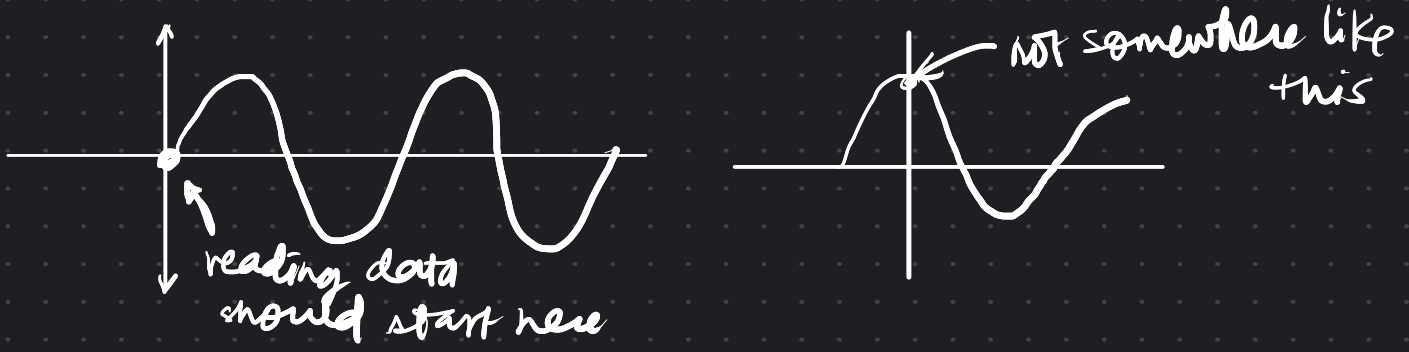

imp
remember

SFD: Start Frame Delimiter
DA: Destination Physical Address (MAC)
SA: Source Physical Address (MAC)
length: of the frame — as length is given, there's no need for ED

Preamble: 101010...10 (7 Bytes)

SFD: 10101011 (1B)

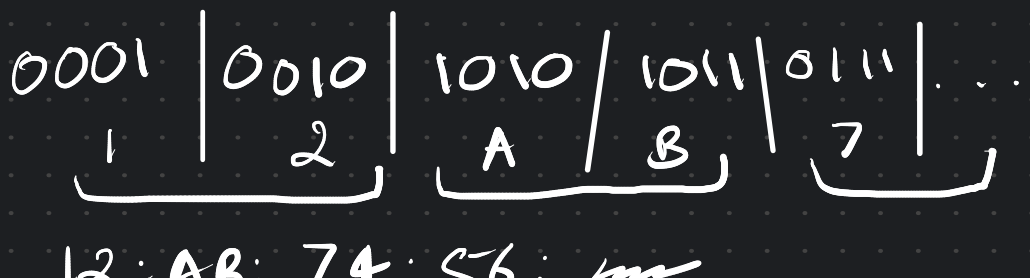
Preamble and SFD are used for synchronization and indicating start of frame.




DA and SA are MAC addresses (48 bit #s)
Represented using the hexadecimal # system.

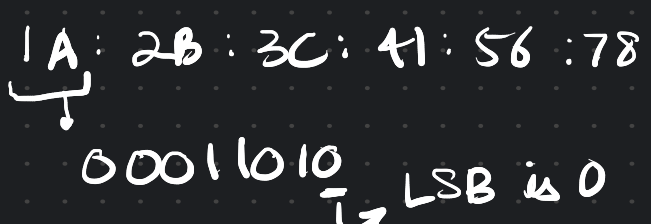
E.g. A MAC address is 000101001010101101 11...48 bits.
Then it'll be divided into parts of 4

then grouped in pairs

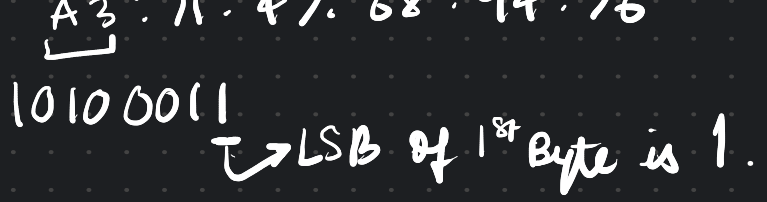



Types of MAC addresses:

- unicast: LSB of 1st byte is 0




- multicast: LSB of 1st byte is 1




- broadcast: all 1's

FF:FF:FF:FF:FF:FF

every host will get a unicast address




say we want to create a group of A, C, E
we'll give them a multicast address
say A1:B2:C3:B4:8A:91
packets forwarded to that address will be read by all 3 of them

broadcast address:
if we want to send a message to all the hosts in the network, we use FF:FF:FF:FF:FF:FF as the destination address.

NOTE: broadcast address at I.P. is 255.255.255.255

At the n/w layer if the addr. is 255.255.255.255, then at the DLL it will be translated to FF:FF:FF:FF:FF:FF

So I.P. broadcasting will be translated to MAC broadcasting
(both directed and limited broadcasting)

Ethernet uses CSMA/CD:

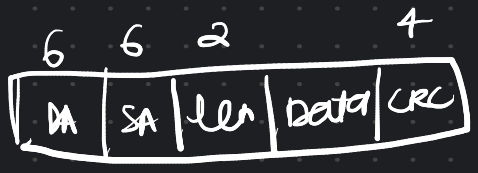
$\therefore L \geq 2 * T_p * BW$

min. frame size

in standard ethernet, Length of frame >= 64 Bytes

max. data in ethernet is 1500 B

a frame has DA, SA, length, data, and crc




min frame size = 64 B

so min data size = 64 - (6+6+2+4) = 46 B

max data size = 1500 B

so max frame size = 1518 B

why is min frame size needed?
to detect collision

why is max frame size needed?
say station A has started sending, B is receiving
then C and D cannot send.
If A never stops, then C and D will never get a chance
(Monopolization)




Disadvantages of Ethernet:

when data is transmitted from both sides, there are a lot of collisions
so it isn't suited for real-time applications (time critical)

say there's a packet and we want to send it by 10:30 (rn it is 10:00)
ethernet cannot guarantee it (due to collisions)

Ethernet is not suitable for interactive applications.
E.g. chatting.
Say we want to send a "hi" (2B). But the min data size = 46 B
We'll have to send unnecessary 44 B

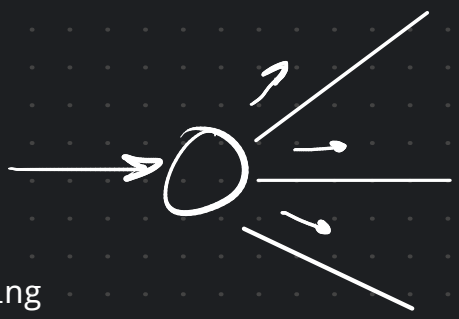
Ethernet is not suitable for client-server applications:
When there's a collision, both hosts have to backoff. No priority to either.
In client-server applications, the server should get higher priority.

Token ring was used to overcome the disadvantages of ethernet.

Switching:

switch could refer to the concept or the device
switch (concept): anything that connects multiple devices
switch (device): device working at DLL

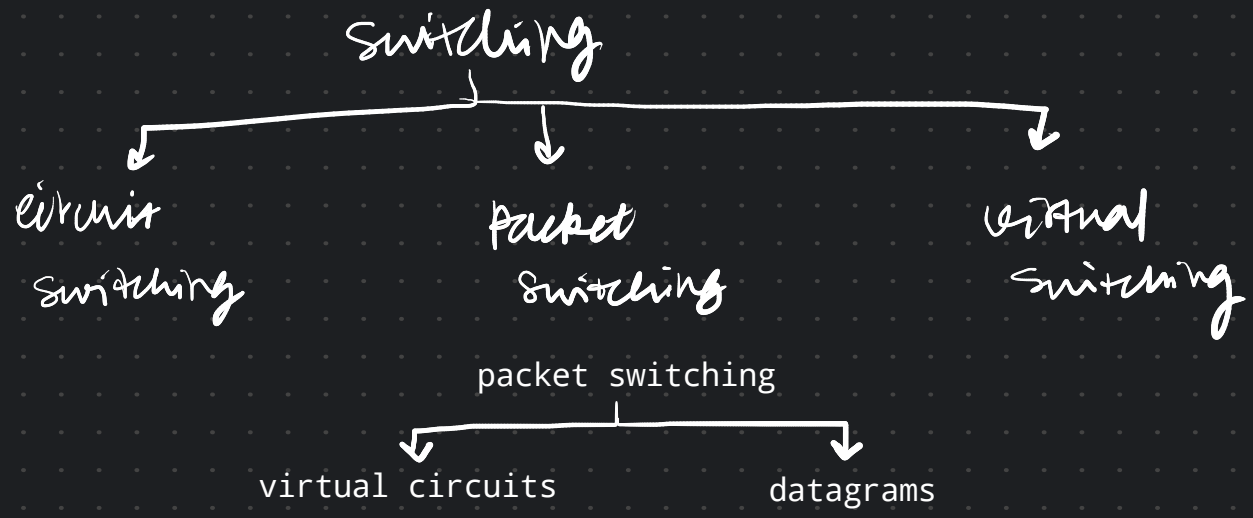
sometimes a router, gateway, bridge, switch,
all of these are called switches in general
switch is a specific device as well



what is switching?

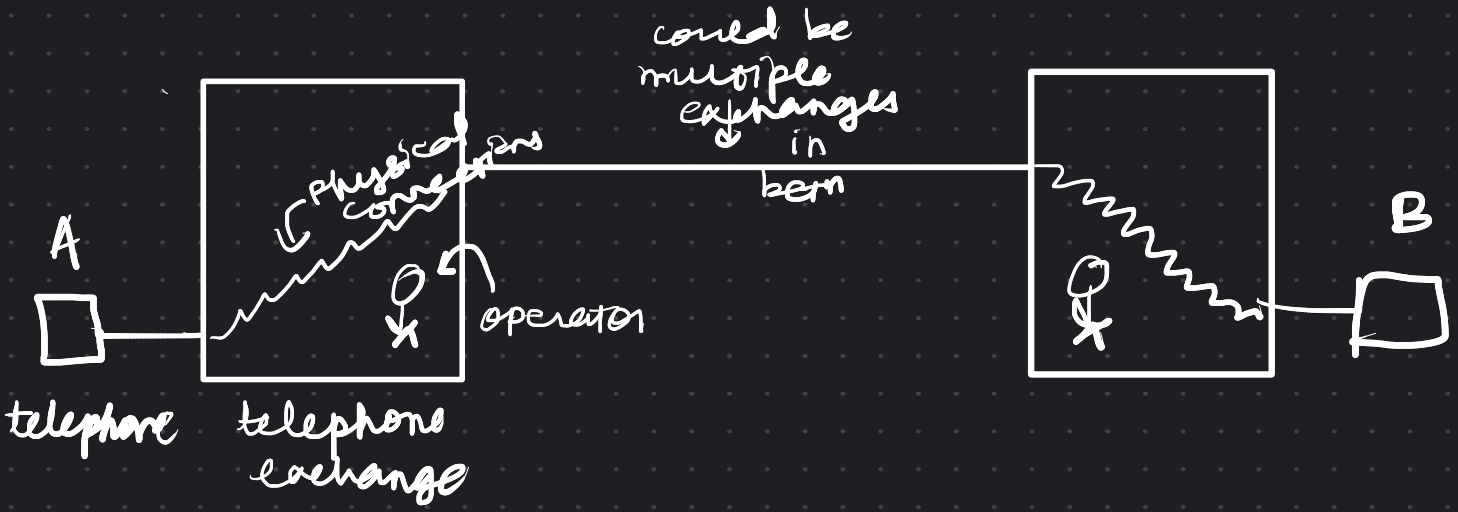
in case of multiple outgoing links,
which one to send to?
making this decision is called switching

a router will do switching using a routing table
building the routing table is called routing
but using the routing table is called switching



Circuit Switching (1980s):

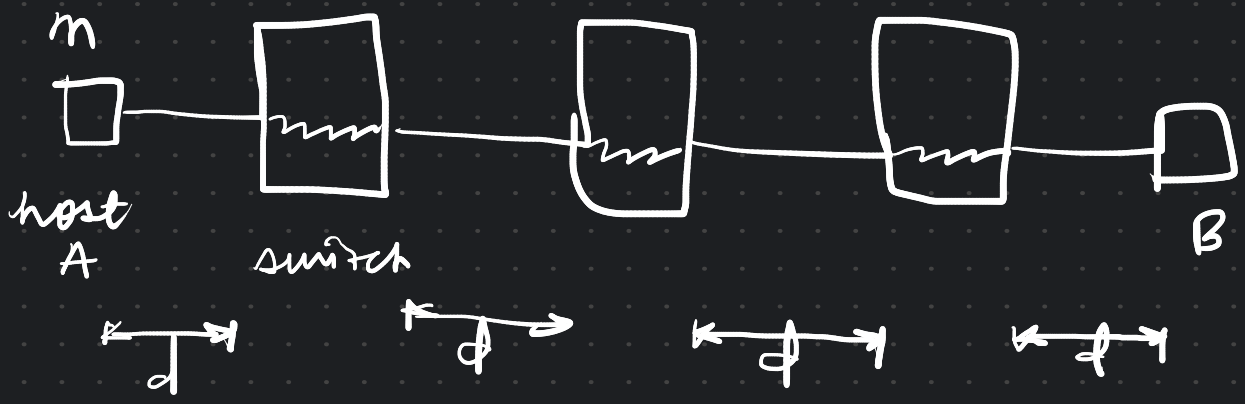
There were telephone exchanges. Many telephones were connected to an exchage, and the exchanges were connected to one another.




say A wants to talk to B. A would contact the operator and both operators would decide time accd. to when a line is available.

The operators could listen to the call as well.
For emergencies, there were trunk calls.

---



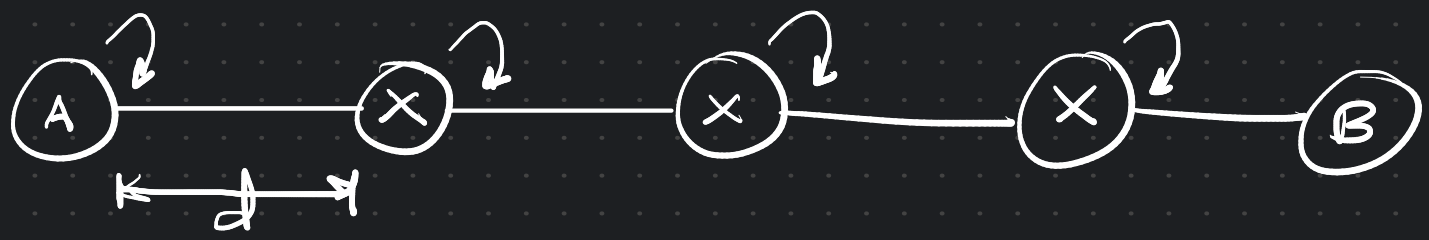

say A wants to send a message of size m to B
Bandwidth = B
distance betn each node = d
velocity = v
# of nodes = x
one transmission is required, the message will go to the end.

total time = setup time (to set up physical links) + transmission time + tear down time (to remove physical links)

$$\frac{m}{B} + \frac{xd}{v}$$

Packet Switching:

There isn't any need of physical intervention. Multiplexing is done programmatically.




there are 4 hops, so 4 transmission delays

total time = $x\frac{m}{B} + x\frac{d}{v}$

A switch gets the packet. It stores it and forwards it.
(store and forward)
Thus the extra transmission delay.

In circuit switching,
total time = setup time + m/B + xd/v + teardown time
In packet switching,
total time = xm/B + xd/v

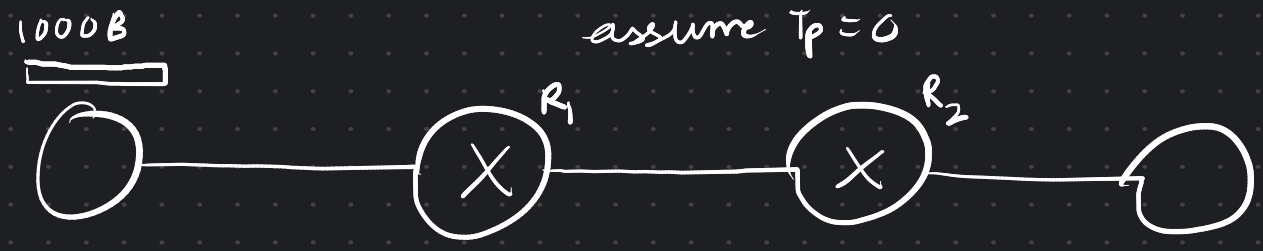
Set-up time and teardown time is extra in circuit switching.
(x-1)m/B is extra in packet switching.

Which is better?
If the message size is very high, then circuit switching is better.
(as there's just a single wire)

If message size is too small, then packet switching is better.
Setup cost is very high for just a small message.

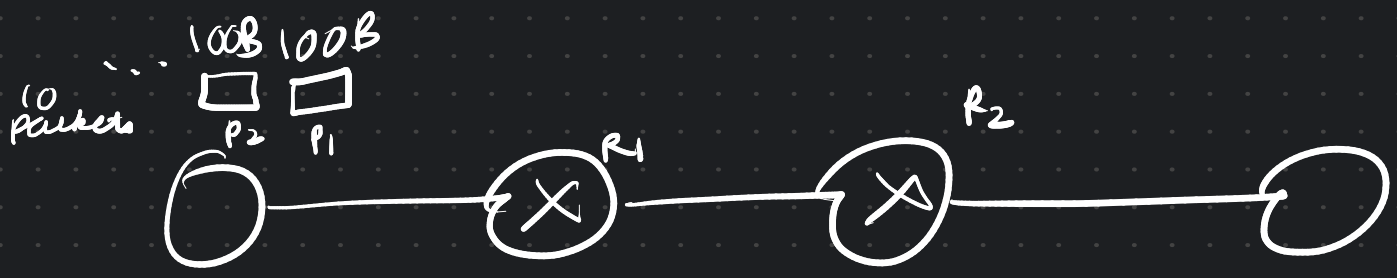
Packetisation:

When the message is too big, we break it into multiple packets.
It improves efficiency of networks.




at t = 0 ms -> transmission starts
at t = 10 ms -> R1 receives 100 B
at t = 20 ms -> R1 receives 200 B
at t = 30 ms -> R1 receives 300 B
...
at t = 100 ms -> R1 receives 1000 B

during all of this time, R1 and R2 are idle




at t = 0 ms -> p1 starts
at t = 10 ms -> R1 receives p1 and R1 starts transmitting p1
at t = 20 ms -> R1 receives p2 and R2 receives p1
at t = 30 ms -> R1 receives p3, R2 receives p2, p1 reaches end
...

routers need full packet before they start transmitting, so they'll wait for the entirity of the packet to reach
in case of small packets, routers can transmit early
but in case of big packets, they'll keep waiting

so we do packetization to make smaller packets.

packetization does a sort of pipelining.

Case 1: Data = 1000 B, BW = 1 MBps, header = 100 B.
Assume Tp = 0.




packet size = data + header = 1100 B

transmission is happening 3 times.

$$T_t = \frac{L}{B} = \frac{1100}{10^6} = 1.1 \times 10^{-3}\ s = 1.1\ ms$$

$$TT = 3 * T_t = 3.3\ ms$$

we could divide 1000 B data into 5 packets (packetization)

Case 2: # of packets = 5, data in each packet = 1000/5 = 200 B
BW = 1 MBps, header = 100 B

packet size = 200 B + 100 B = 300 B




$$T_t = \frac{300}{10^6} = 0.3\ ms.$$

at t = 0.3 ms, p1 will be at switch 1

at t = 3*0.3 ms, packet 1 will be at R
2nd will be at switch 2
3rd will be at switch 1
4th and 5th will be at S

rest will take 4 * 0.3 ms in total to come out

so total time = 3*0.3 + 4*0.3 = 2.1 ms

case 3: # of packets = 10, so data in each = 1000/10 = 100 B
BW = 1MBps, header = 100 B

packet size = 200 B

$T_t = \frac{200}{10^6} = 0.2$ ms




for packet 1, 3*0.2 ms to come out
for all others, 9*0.2 in total

total time = 2.4 ms

case 1: 3.3 ms, 1 packet

case 2: 2.1 ms, 5 packets

case 3: 2.4 ms 10 packets

so excess packetization would increase the time as each packet would take that many times * Tt.

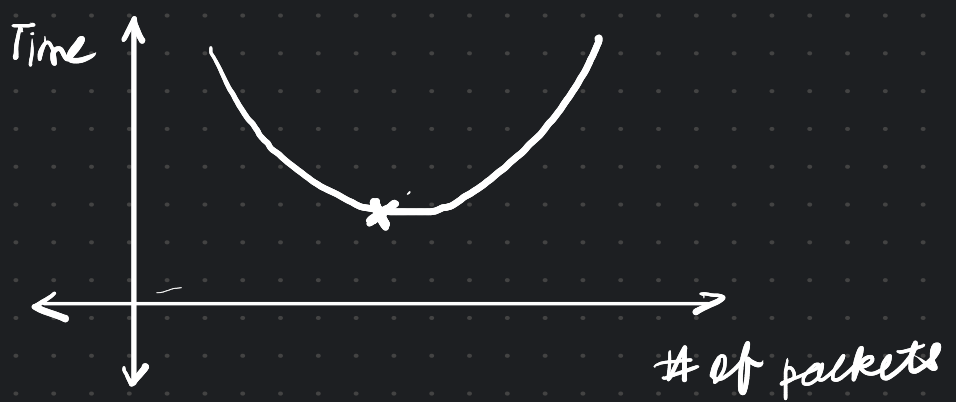
case 4: # of packets = 20. so data in each = 50 B, BW = 1 MBps, header = 100 B
packet size = 150 B

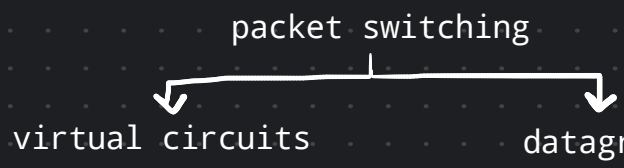
$T_t = 0.15$ ms.

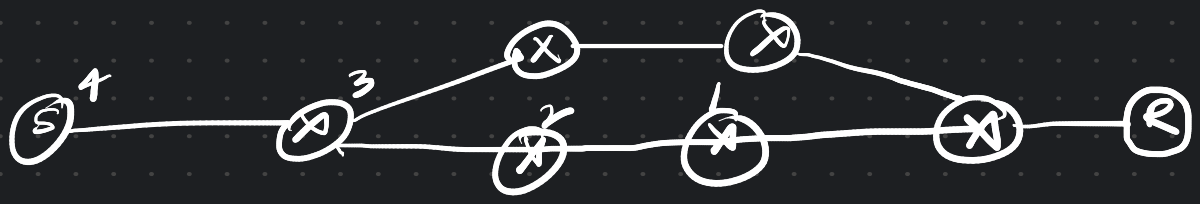



for first 0.15*3 ms to reach R
for rest 19*0.15 ms in total to reach R

total time = 3.3 ms






In virtual circuits, whenever first packet is going,
rest of the packets follow the first one.
When the first packet is going, it'll ask routers to reserve
bandwidth, buffer, and CPU for the following packets
so that the packets never get discarded.
That tunnel like thing formed is a virtual circuit.

In datagrams, all packets go individually. They may take different routes.
If the sender sends a packet, the packet may be discarded
as bandwidth, buffer, or CPU may not be available.

Telephone calls use virtual circuits.
In telephone calls, when you hear the first ring, the first packet will go.
The virtual circuit will be established.
When the first packet reaches that end, resources are reserved. So we pay for that time.

E-mail uses datagrams.
For packets, resources aren't reserved. So email charges for the amount of data transferred, not the time.
Packets may take different routes, so slow. Packets may get lost.
Out of data packets are possible.
Internet calls use datagram. So these are lower in quality.
VOIP: Voice Over Internet Protocol

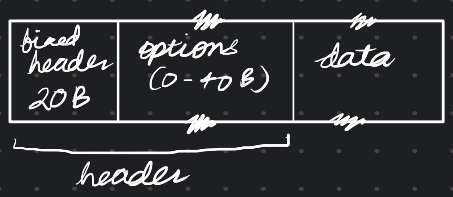
IP uses datagram.

Virtual Circuits and Datagrams:

| Virtual Circuit | Datagram |
| --- | --- |
| - all packets follow 1st packet. so the same route is taken. e.g. phone calls | - all packets go independently. e.g. data calls (internet calls) |
| - only first packet needs a header, rest follow the first so they don't | - every packet should contain a header |
| - connection-oriented (reservations for buffer, bw, cpu) | - connection-less no such reservations |
| - entire data follows same path | - same path may not be followed |
| - highly reliable | - not reliable |
| - high cost (rent for resources) | - less cost (pay per use/ data) |
| - used in ATM networks (asynchro-nous transfer mode) | - used in internet protocol |

message switching: sending message without dividing into packets
(not imp)

Internet Protocol (IP):




IPV4 Header:

| | | | | | | | | |
|---|---|---|---|---|---|---|---|---|
| version (4) | HL (4) | type of service (8) | total length (16) | | | | | 4B |
| identification (16) | | | 0 (1) | df (1) | mf (1) | fragment offset (13) | | 4B |
| TTL (8) | protocol (8) | | header checksum (16) | | | | | 4B |
| source ip (32 bits) | | | | | | | | 4B |
| dest ip (32 bits) | | | | | | | | 4B |
| options (0-40 Bytes) | | | | | | | | 0-40B |
| data | | | | | | | | |

total: 20 - 60 B

HL: header length
It is required because header is 20B and along with it there are options (0 to 40B)
So HL can be 20 to 60 B.
Size of header length field: 4 bits
so max it can represent = 1111 i.e. 15
but max header len is 60 B
so if we want to represent 60, we use a scaling factor of 4
meaning, if HL is 20 B, then HLF (header length field) will be 0101 i.e. 4 (as 20/4=5)

we make header size a multiple of 4 by padding so that we can do this

DF: do not fragment (1 or 0)

MF: more fragments

TTL: time to live

version: 4 bits (0-15)

currently V4 is going on. V5 was invented, but failed in testing.
We are moving on to V6

identification #: 16 bits (0-65535 #s are possible)

for datagrams, every ip packet is numbered

NOTE: At IP, data grams are numbered. In TCP, bytes are numbered.

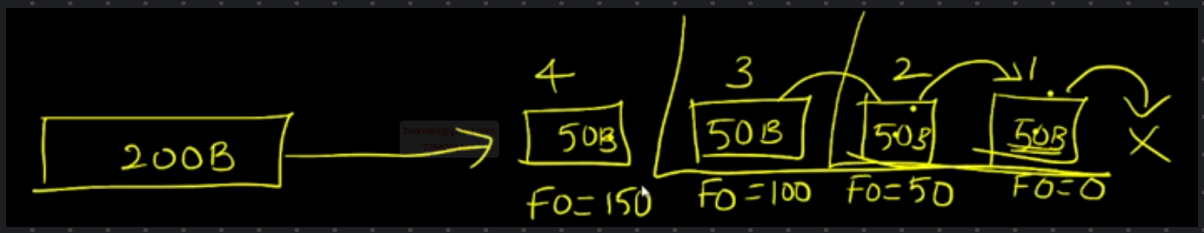
DF: do not fragment (1 bit)

say network N1 wants to send 200 B packet but in receiver N2,
only 100 B max size is allowed. The router would break a packet into multiple smaller ones (fragmentation)
If DF = 1, then the router is asked not to fragment the packet.
The router will discard the packet and send a message to the sender N1.
Then the host will themselves break the packet into 100 B and 100 B or decide for themselves.

MF: more fragments (1 bit)

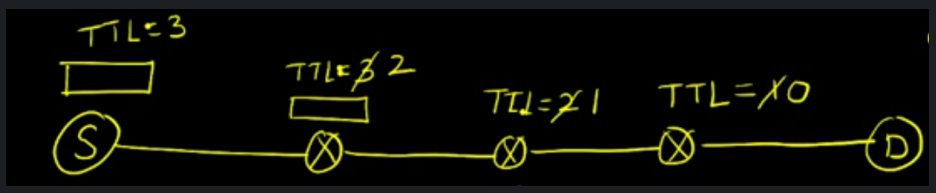
say a packet is 200 B. Router divides into 50 B * 4.
MF = 1 indicates that there are more fragments following the current packet.
For the last packet reaching the dest, MF = 0
Rest, MF = 1

Fragment Offset: # of Bytes ahead of a particular fragment




TTL: Time to Live (8 bits)

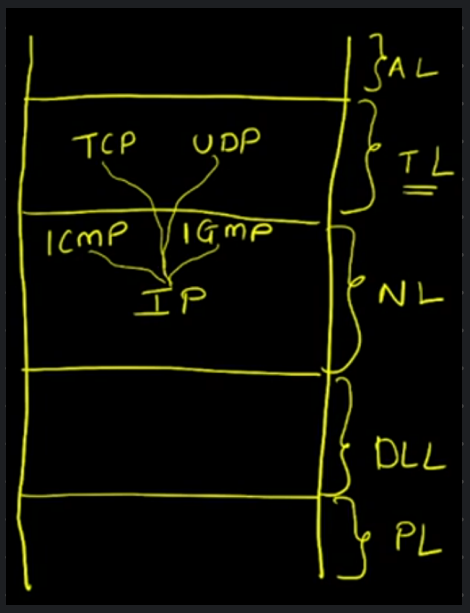
when ttl becomes 0, the packet is discarded



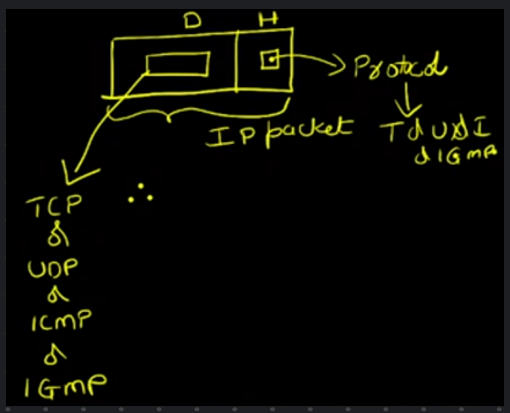

the ttl can become 0 at destination, that's fine. it'll be accepted.
or if ttl > 0 at destination, it is accepted. if 0 before dest, rejected.

TTL is used to prevent infinite looping (due to errors)
of packets in networks.
If a packet is stuck in a loop, it'll be discarded after its TTL becomes 0

say you go from India to London in 1 hop, London to US in 1 hop,
US to Japan in 1 hop.
So in 3-4 hops, you can cover the world.
Then TTL would either be 8 or 16. (or at max 32)
Generally, TTL is set to 8.




Protocol (8 bits):
At transport layer, TCP and UDP work.
At the network layer, IP works.



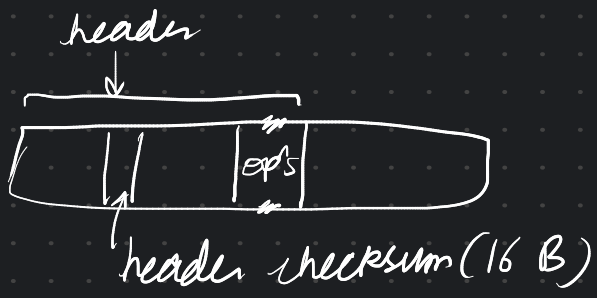

At a router, a packet may be discarded if there is congestion
(not enough resources).

Say there's a router and there's a packet with protocol TCP.
At the router, there'll be a buffer. Say there are datagrams in.
One UDP, one ICMP, one IGMP, one TCP. Say the buffer is now full.
So which packet to discard? That depends on priority.
TCP > UDP > IGMP > ICMP - priority order
ICMP packet will be discarded and the TCP packet will take its place.

Why such a priority order?
If the TCP packet is discarded, it'll be retransmitted.
UDP may or may not be retransmitted.
IGMP and ICMP aren't retransmitted.

Say all the packets in the buffer are TCP, then the incoming TCP packet
will be discarded. The old ones will be given a higher priority.

Header checksum:




The header will be divided into 16 bit #s. Then checksum is calculated. In order to calculate the value of checksum, considering that the checksum itself is one of the 16 bit #s, we initially keep the checksum = 00000..0

Why is checksum only calculated for the header, and not the body?
If we calculate checksum on the entire packet, the packet will go to the next router. At the router, the header will change. Some of the fields will change. So every router would have to calculate checksum of the entire body. To reduce load on those routers, only checksum for the header is calculated.
The entire body checksum would require more computation.

At every router, header checksum will be calculated.
(body checksum will be a burden)

What fields may change?
- TTL will change
- Fragment offset, MF, total length may change
- options may change

Identification #s go from 0, 1, 2, .., 65536 (as the field has 16 bits) after that, the #s are reused (wraparound). On wraparound, the previous packet that used the same # would be dead.
Anyways it takes a lot of time to use up those many #s.

An IP address is divided into NID and HID parts.

| NID | HID | gives | can be used as |
|---|---|---|---|
| valid | valid | valid IP address | SIP, DIP |
| valid | all 0's | NID | Neither |
| valid | all 1's | DBA | Only DIP |
| all 1's | all 1's | LBA | Only DIP |
| all 1's | all 0's | Subnet Mask | Neither |
| all 0's | valid | host within a n/w | SIP, DIP |
| all 0's | all 0's | i don't have an ip | Only SIP |
| 127 | valid | loop back address | Only DIP |

---

Options:

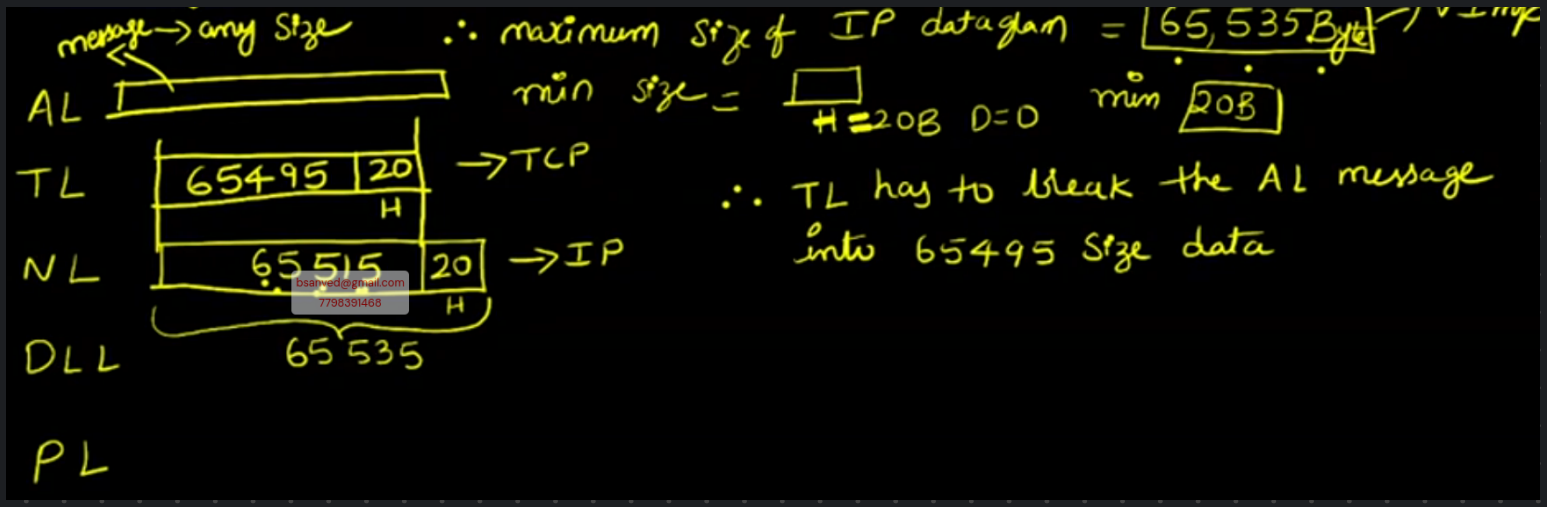
i. Record Route: With the record route option enabled, every router will put its IP address in the IP header. The destination will see the route, not the source. For source to see the route, we use trace route.

ii. Source Routing: With the record route option enabled, the source can specify the entire route to be taken.
Source routing can be strict or loose.
In strict source routing, all routers
are specified and the exact path specified with be taken.
In loose source routing, few of the routers are specified and the path between those is decided on the go.

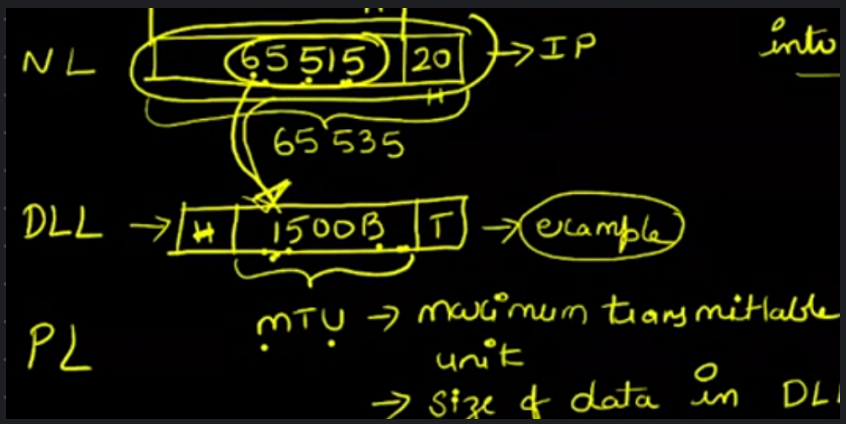
Size of options is 40 B at max. Size of the IP address is 4B.
So in both record route and source routing, the max IP addresses we can put in options is 40/4 = 10. But we cannot actually put 10, as there has to be space to specify the type of options.
So at max, 9 IP addresses can be put in there.

Total length field:
The total length field is a 16 bit #.
Max size of IP datagram = 65,535 B
Min size = 20 B header + data = 20 B

In the network layer, header is 20B. Total 65535, so left 65515.
In this 65515 B, the transport layer part has to fit in.
So the TCP packet is 65515B. The transport layer header is 20B.
65495B is left.
The application layer message can be of any size.
So the transport layer has to break that message into data of size 65495.
(Segmentation)

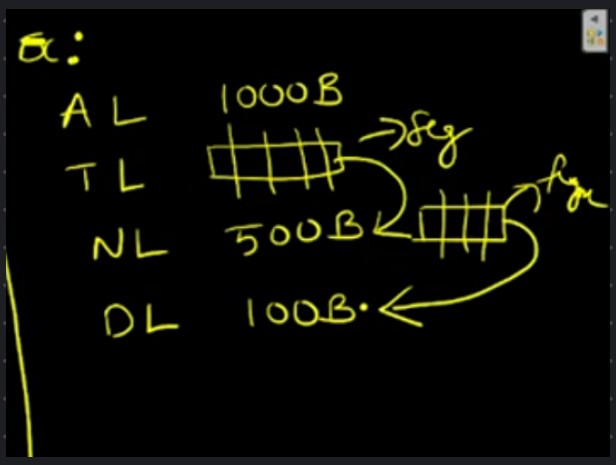



There'll be a maximum transmittable unit (MTU) at the DLL. The datagram of the NL should fit in that much space.
MTU: Size of data in DLL.




The NL has to break the datagram to fit into a DLL frame. (Fragmentation)

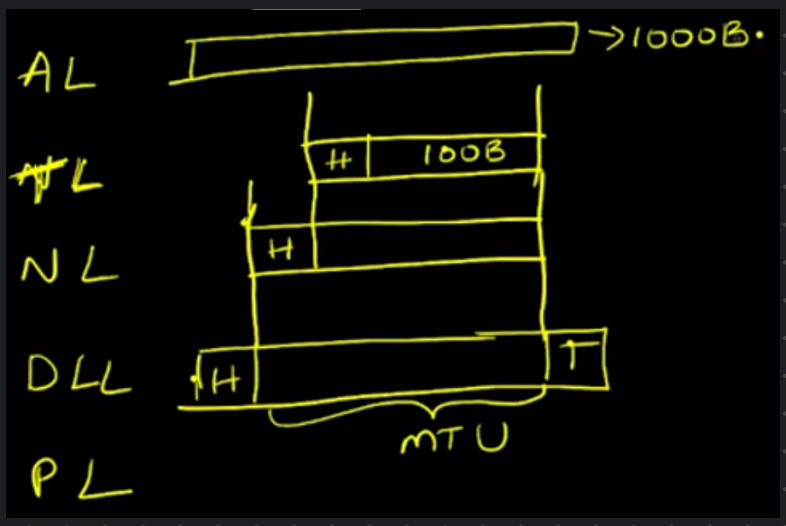
So TL does segmentation,
NL does fragmentation.



But TL can do something clever.

Here if TL breaks AL message into 100 B parts, then the data will directly fit in the DLL frame without fragmentation.

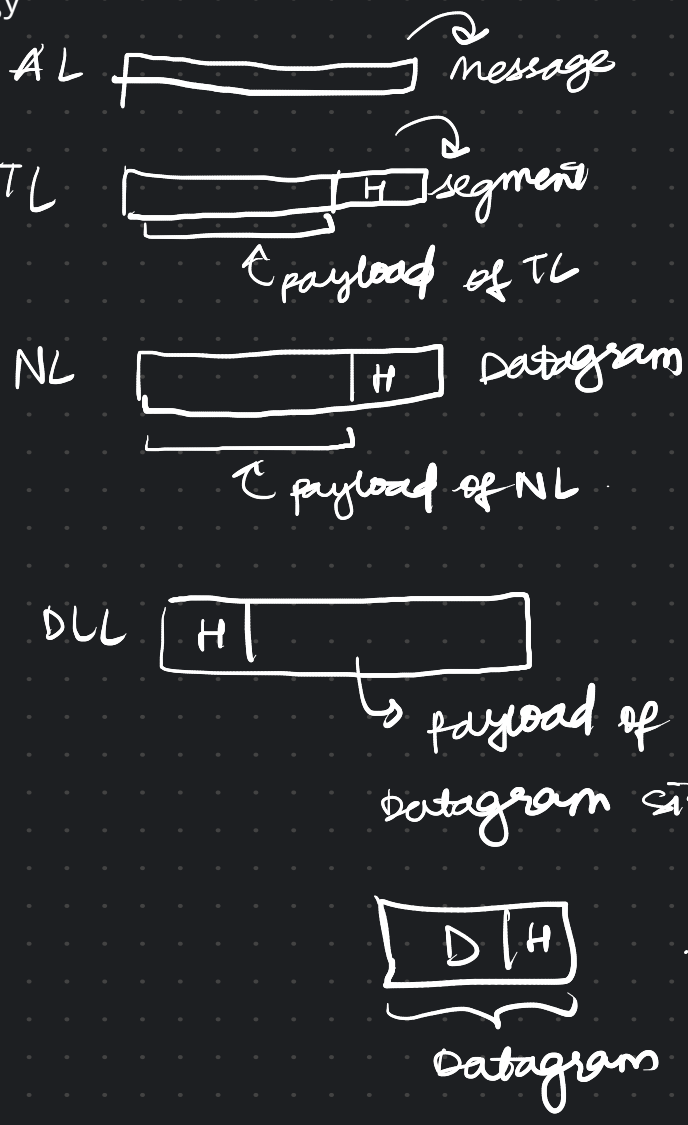
So at source, only segmentation is needed, not fragmentation.




If we do segmentation wisely, there is no need of fragmentation at source.
But fragmentation will happen at the routers.

---

Terminology





MTU = 500B

↳ Payload of DLL = 500 B

↳ IP datagram size = 500 B

So header size + data = 500 B

So max. data = 500 - 20 = 480 B.

https://en.wikipedia.org/wiki/Internet_Protocol_version_4

**IPv4 header format**

| *Offsets* | Octet | 0 | | | | | | | | 1 | | | | | | | | 2 | | | | | | | | 3 | | | | | | | |
|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|---|
| Octet | Bit | 0 | 1 | 2 | 3 | 4 | 5 | 6 | 7 | 8 | 9 | 10 | 11 | 12 | 13 | 14 | 15 | 16 | 17 | 18 | 19 | 20 | 21 | 22 | 23 | 24 | 25 | 26 | 27 | 28 | 29 | 30 | 31 |
| 0 | 0 | Version | | | | IHL | | | | DSCP | | | | | | ECN | | Total length | | | | | | | | | | | | | | | |
| 4 | 32 | Identification | | | | | | | | | | | | | | | | Flags | | | Fragment offset | | | | | | | | | | | |
| 8 | 64 | Time to Live | | | | | | | | Protocol | | | | | | | | Header checksum | | | | | | | | | | | | | | | |
| 12 | 96 | Source address | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 16 | 128 | Destination address | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 20 | 160 | Options (if IHL > 5) | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| ⋮ | ⋮ | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |
| 56 | 448 | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | | |

**Version**

The first header field in an IP packet is the four-bit version field. For IPv4, this is always equal to 4.

**Internet Header Length (IHL)**

The IPv4 header is variable in size due to the optional 14th field (options). The IHL field contains the size of the IPv4 header; it has 4 bits that specify the number of 32-bit words in the header. The minimum value for this field is 5,[32] which indicates a length of 5 × 32 bits = 160 bits = 20 bytes. As a 4-bit field, the maximum value is 15; this means that the maximum size of the IPv4 header is 15 × 32 bits = 480 bits = 60 bytes.

**Differentiated Services Code Point (DSCP)**

Originally defined as the type of service (ToS), this field specifies differentiated services (DiffServ).[33] Real-time data streaming makes use of the DSCP field. An example is Voice over IP (VoIP), which is used for interactive voice services.

**Explicit Congestion Notification (ECN)**

This field allows end-to-end notification of network congestion without dropping packets.[34] ECN is an optional feature available when both endpoints support it and effective when also supported by the underlying network.

**Total Length**

This 16-bit field defines the entire packet size in bytes, including header and data. The minimum size is 20 bytes (header without data) and the maximum is 65,535 bytes. All hosts are required to be able to reassemble datagrams of size up to 576 bytes, but most modern hosts handle much larger packets. Links may impose further restrictions on the packet size, in which case datagrams must be fragmented. Fragmentation in IPv4 is performed in either the sending host or in routers. Reassembly is performed at the receiving host.

**Identification**

This field is an identification field and is primarily used for uniquely identifying the group of fragments of a single IP datagram. Some experimental work has suggested using the ID field for other purposes, such as for adding packet-tracing information to help trace datagrams with spoofed source addresses,[35] but any such use is now prohibited.[36]

**Flags**

A three-bit field follows and is used to control or identify fragments. They are (in order, from most significant to least significant):

- bit 0: Reserved; must be zero.[a]
- bit 1: Don't Fragment (DF)
- bit 2: More Fragments (MF)

If the DF flag is set, and fragmentation is required to route the packet, then the packet is dropped. This can be used when sending packets to a host that does not have resources to perform reassembly of fragments. It can also be used for path MTU discovery, either automatically by the host IP software, or manually using diagnostic tools such as ping or traceroute. For unfragmented packets, the MF flag is cleared. For fragmented packets, all fragments except the last have the MF flag set. The last fragment has a non-zero Fragment Offset field, differentiating it from an unfragmented packet.

**Fragment offset**

This field specifies the offset of a particular fragment relative to the beginning of the original unfragmented IP datagram. The fragmentation offset value for the first fragment is always 0. The field is 13 bits wide, so that the offset can be from 0 to 8191 (from ($2^0$ – 1) to ($2^{13}$ – 1)). Fragments are specified in units of 8 bytes, which is why fragment length must be a multiple of 8.[37] Therefore, the 13-bit field allows a maximum offset of ($2^{13}$ – 1) × 8 = 65,528 bytes, with the header length included (65,528 + 20 = 65,548 bytes), supporting fragmentation of packets exceeding the maximum IP length of 65,535 bytes.

**Time to live (TTL)**

An eight-bit time to live field limits a datagram's lifetime to prevent network failure in the event of a routing loop. It is specified in seconds, but time intervals less than 1 second are rounded up to 1. In practice, the field is used as a hop count—when the datagram arrives at a router, the router decrements the TTL field by one. When the TTL field hits zero, the router discards the packet and typically sends an ICMP time exceeded message to the sender.

The program *traceroute* sends messages with adjusted TTL values and uses these ICMP time exceeded messages to identify the routers traversed by packets from the source to the destination.

**Protocol**

This field defines the protocol used in the data portion of the IP datagram. IANA maintains a list of IP protocol numbers.[38]

**Header checksum**

The 16-bit IPv4 header checksum field is used for error checking of the header. When a packet arrives at a router or its destination, the network device calculates the checksum of the header including the checksum field. A value of 0xFFFF is expected. If a different result is obtained, the device discards the packet. Errors in the data portion of the packet are handled separately by the encapsulated protocol. Both UDP and TCP have separate checksums that apply to their data.

When a packet arrives at a router, the router decreases the TTL field in the header. Consequently, the router must calculate a new header checksum.

The checksum field is the 16-bit ones' complement of the ones' complement sum of all 16-bit words in the header. For purposes of computing the checksum, the value of the checksum field is zero.

**Source address**

This 32-bit field is the IPv4 address of the sender of the packet. It may be changed in transit by network address translation (NAT).

**Destination address**

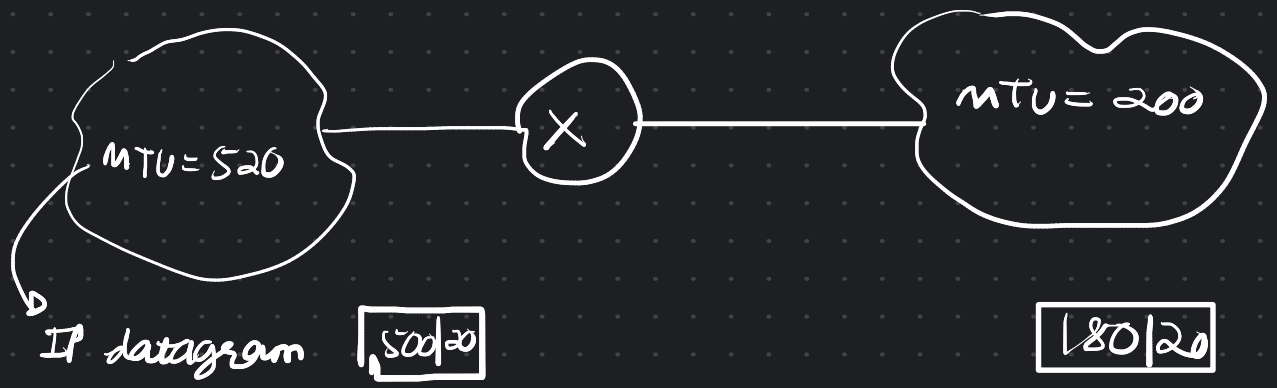
This 32-bit field is the IPv4 address of the receiver of the packet. It may be affected by NAT.

**Options**

The options field is not often used. Packets containing some options may be considered as dangerous by some routers and be blocked.[39] The value in the IHL field must include sufficient extra 32-bit words to hold all options and any padding needed to ensure that the header contains an integral number of 32-bit words. If IHL is greater than 5 (i.e., it is from 6 to 15) it means that the options field is present and must be considered. The list of options may be terminated with the option EOOL (End of Options List, 0x00); this is only necessary if the end of the options would not otherwise coincide with the end of the header. The possible options that can be put in the header are as follows:

| Field | Size (bits) | Description |
|---|---|---|
| Copied | 1 | Set to 1 if the options need to be copied into all fragments of a fragmented packet. |
| Option Class | 2 | A general options category. 0 is for *control* options, and 2 is for *debugging and measurement*. 1 and 3 are reserved. |
| Option Number | 5 | Specifies an option. |
| Option Length | 8 | Indicates the size of the entire option (including this field). This field may not exist for simple options. |
| Option Data | Variable | Option-specific data. This field may not exist for simple options. |

Fragmentation at Routers:




500B has to be divided into 180B sized fragments.
We'll get 3 fragments.

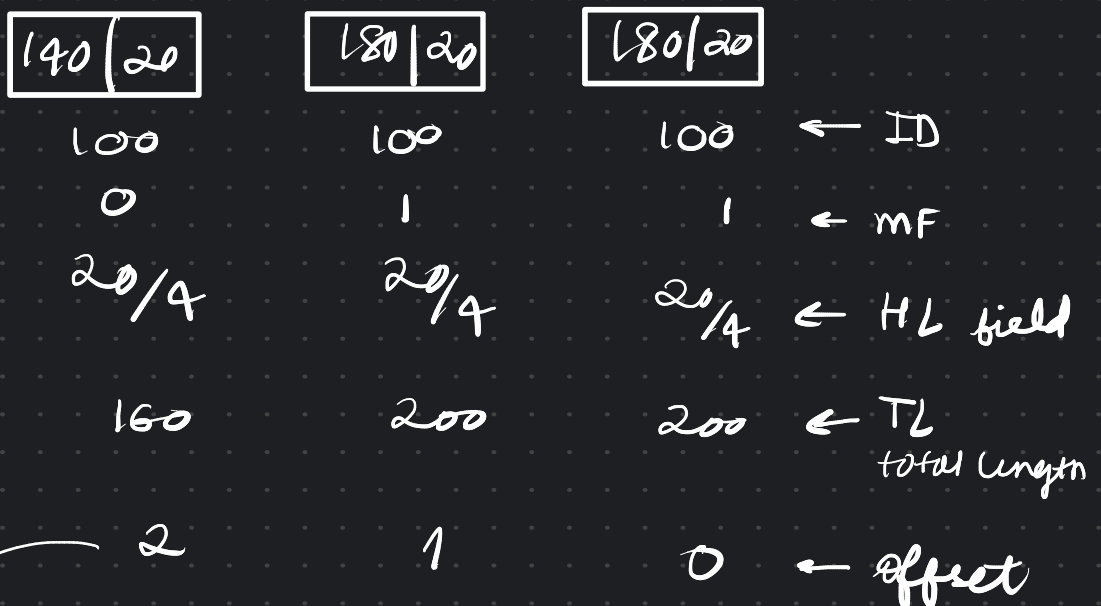



140 + 180 + 180 = 500

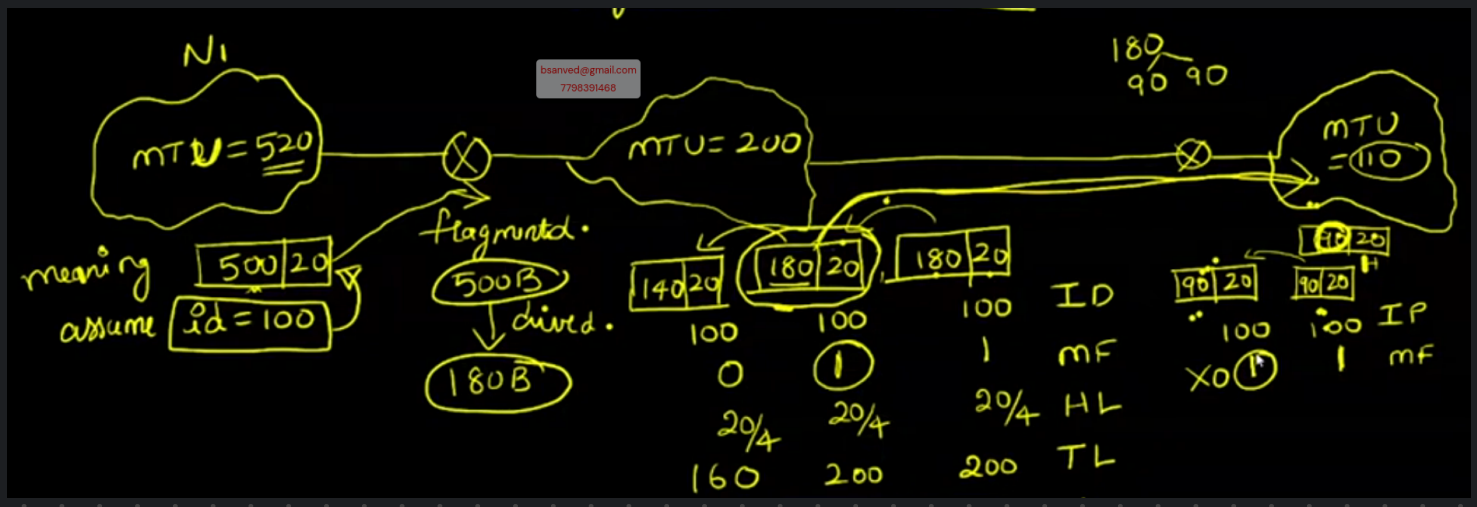
How will the receiver know that all the fragments are of the same datagram?
- identification #
  Say on the original packet = 100. It'll be so on the 3 fragments as well.

How will receiver understand if they've received all the fragments?
- MF

How will receiver order the fragments?
- offset




This method will fail.
Say there is another n/w with MTU = 110 [90|20]
And say the 2nd packet wants to go there.
It'll be fragmented so: [180|20] → [90|20] [90|20]




So there'll be confusion. → This won't be zero as the parent packet has more packets following it.

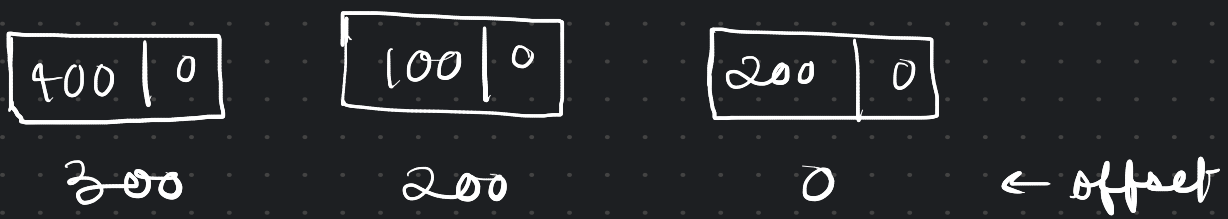
Parent MF = 1
∴ child MF = 1

| 90\|20 | 90\|20 | |
|---|---|---|
| 1 | 1 | MF |
| 20/4 | 20/4 | HL |
| 110 | 110 | TL |
| | | off |

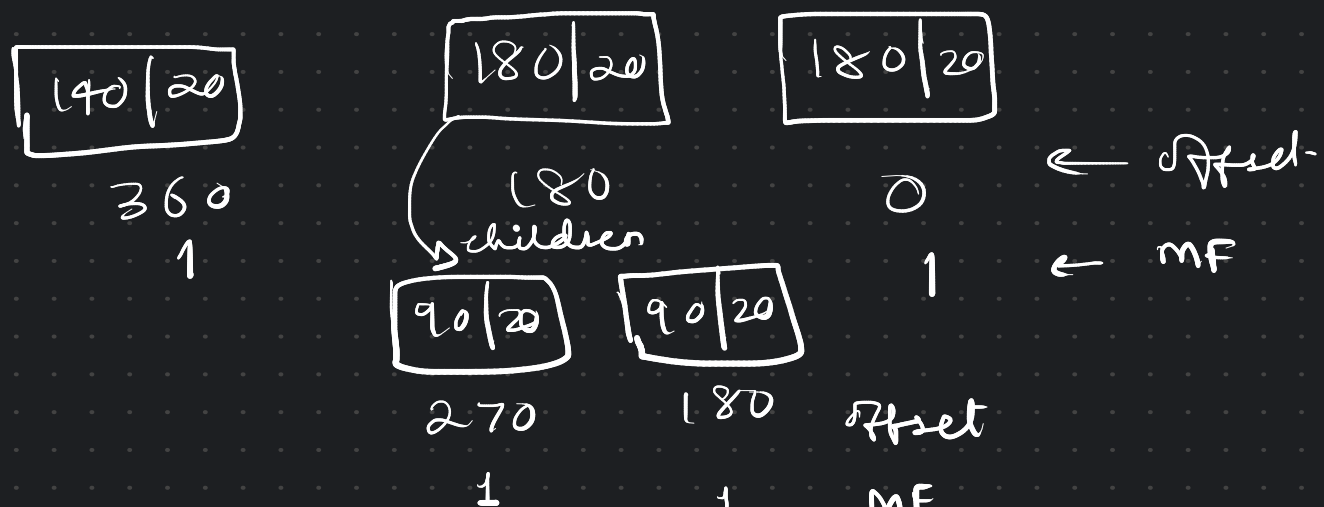
here offsets 0 & 1 cannot be reused ← as already other packets have them.

We need a system for offset that'll hold even after fragmenting multiple times.
So the #s 0, 1, 2, ... won't work

We use the offset as the # of data bytes ahead a fragment. (header not included)

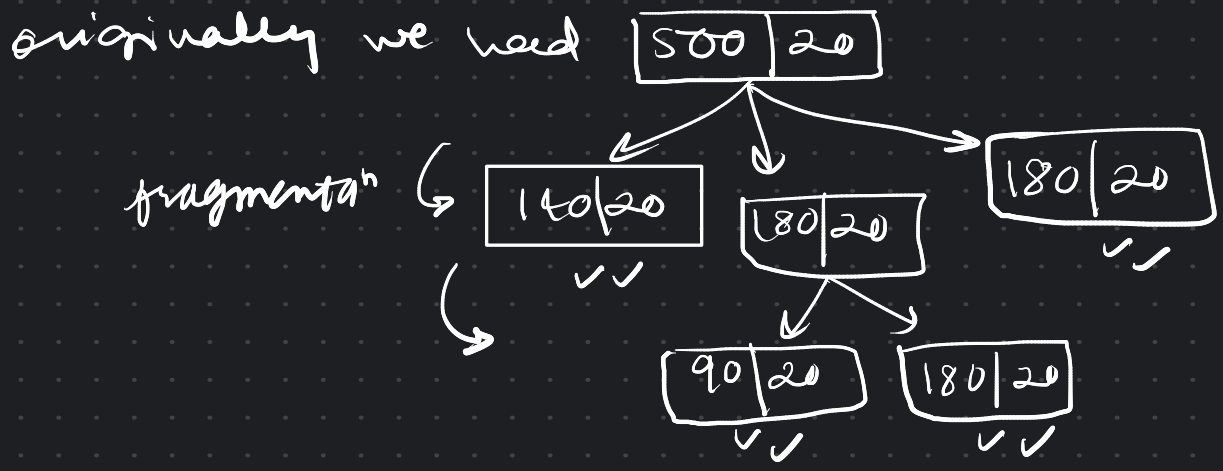



---



How many fragments will the receiver receive?
- 4



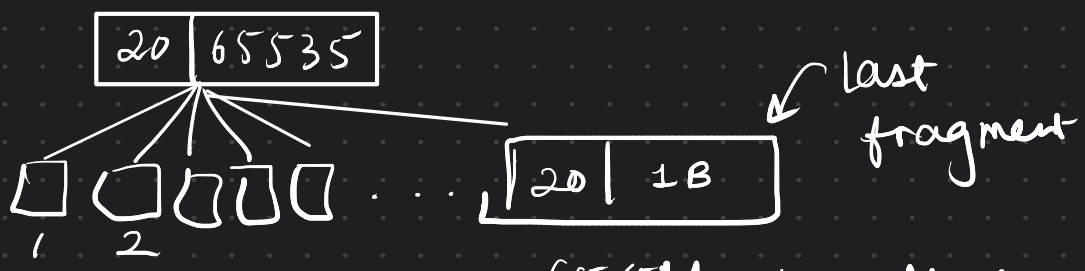




---




| | | | | |
|---|---|---|---|---|
| ID | 100 | 100 | 100 | 100 |
| offset | 0 | 180 | 270 | 360 |
| MF | 1 | 1 | 1 | 0 |
| HL | 5 | 5 | 5 | 5 |
| TL | 200 | 110 | 110 | 160 |

max size of IP packet is 2^16-1 = 65535. 20B header and 65515B data.
say it gets fragmented into a lot of fragments



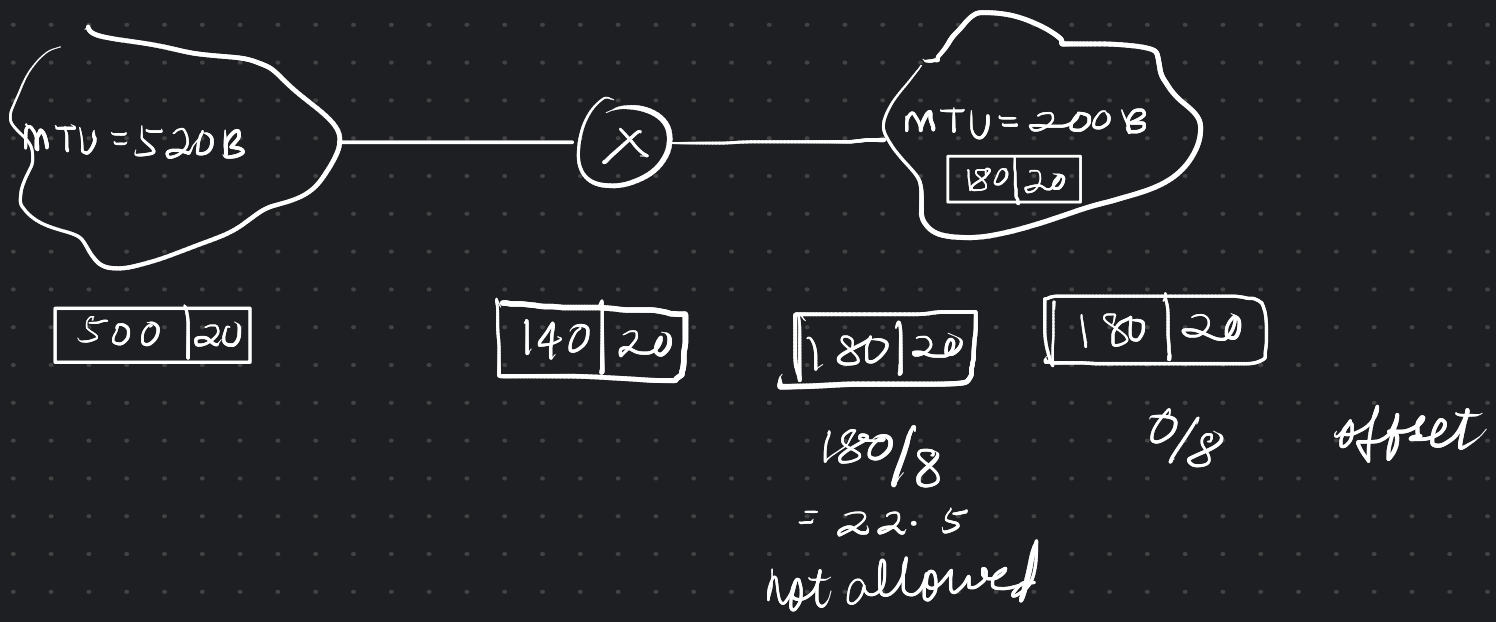

$65514 \approx 2^{16}$

So to store it, we need 16 bits. But offset only has 13 bits.
So we do scaling like in header length field.

$\frac{2^{16}}{2^{13}} = 8$ ← scaling factor

[In HL, Go/15 = 4

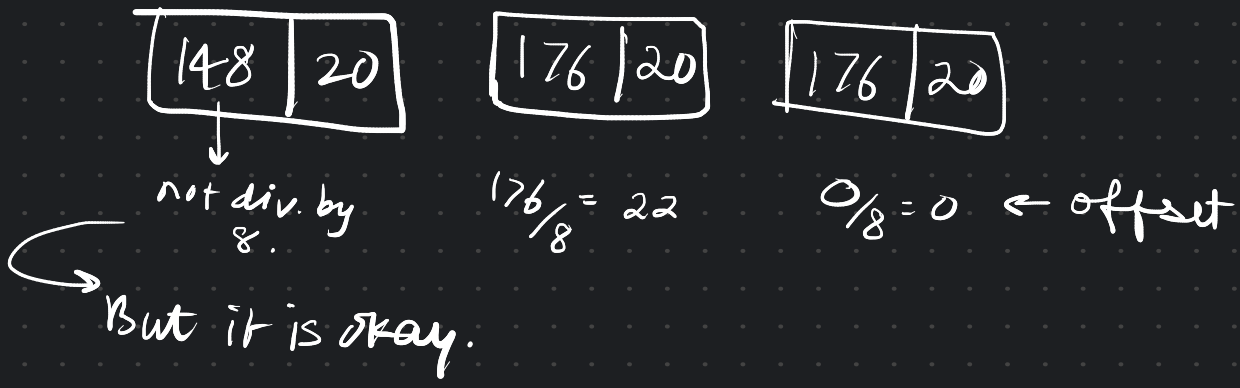
so all the offsets must be multiples of 8.
so what we've done till now is faulty.



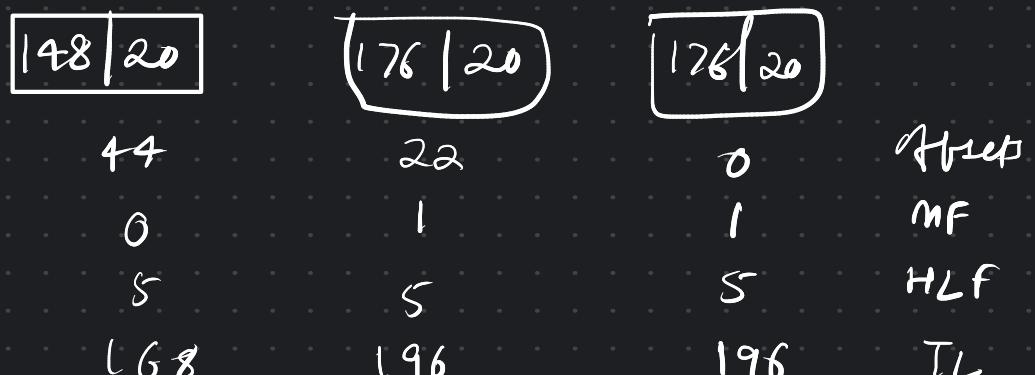

we can't even use padding to make it a multiple of 8, as the datagram would then exceed the MTU.

So instead of adding 4 bytes to make it 184 (a multiple of 8), we'd remove 4 bytes to do so.

A # divisible by 8 just below 180 is 176.

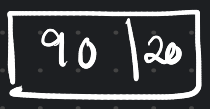



For the last fragment, no other fragment is following it.
So the last fragment bytes need not be divisible by 8.

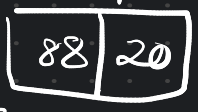



say the middle packet is going to a n/w with MTU = 110

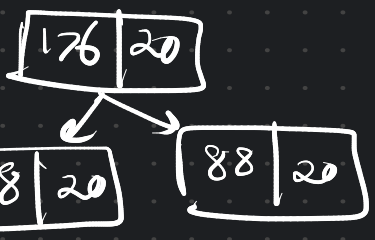
MTU = 110




90 isn't divisible by 8.
So we take




176 - 88 = 88




$\frac{22+88}{8} = 33$

or

$\frac{176+88}{8} = 33$

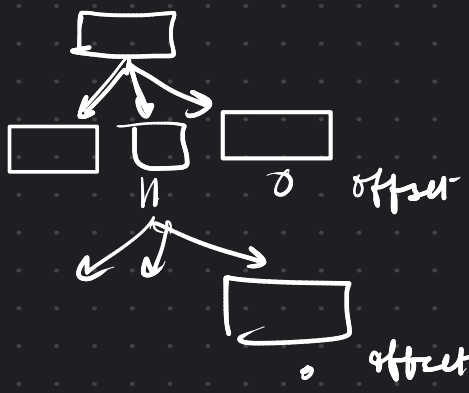
22 ← same as parent's

Offset

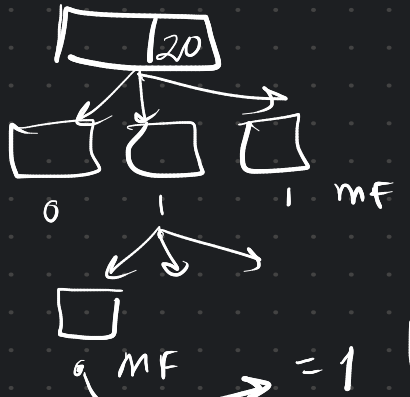
1 1 MF

---

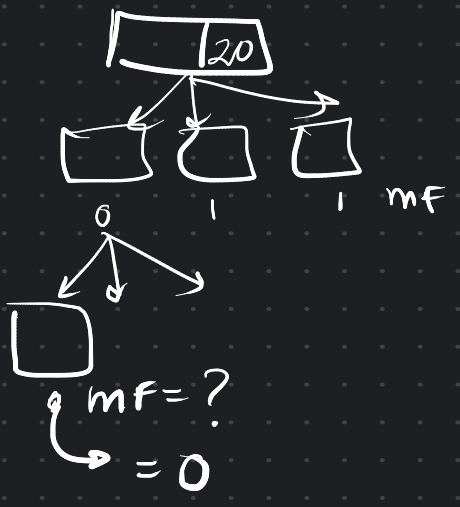
cases:



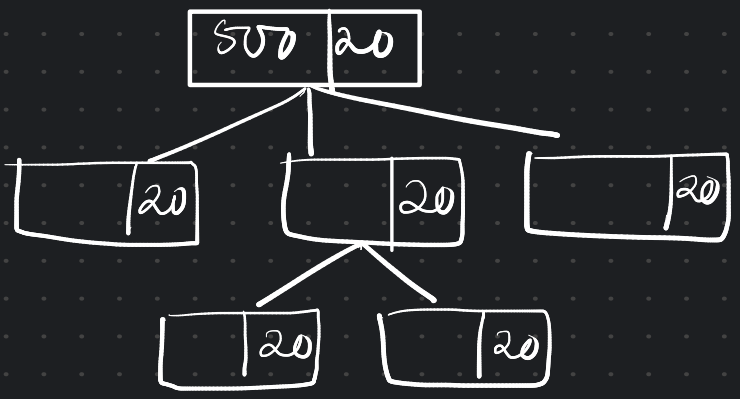

offset = ?
→ 0 (1st child will have same offset as parent)




[MF of last child = MF of parent]



In the previous example,




original packet → 1 header

all the fragments → 4H

overhead = 4 - 1

= 3 headers

= 60 B.

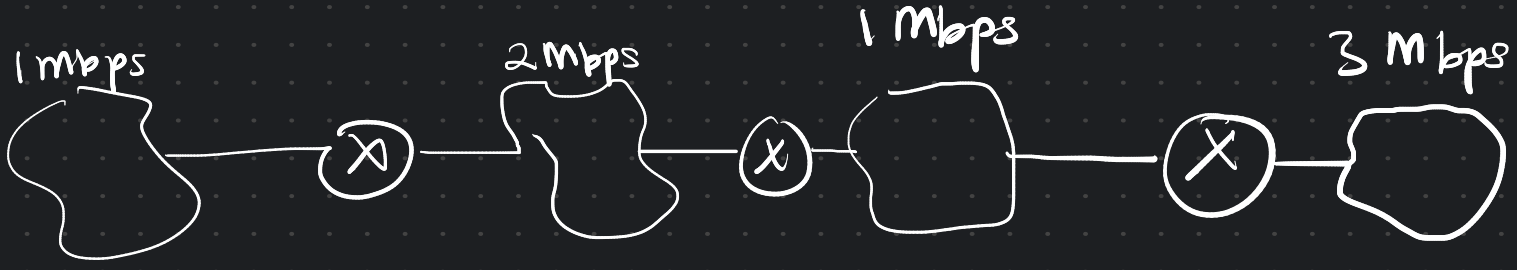
$$\eta = \frac{\text{useful data}}{\text{total data}}$$

$$\eta = \frac{500}{500 + 4*20} \text{ B}$$

useful data doesn't have any headers

effective bandwidth or throughput $= \eta * B$ or BW utilization

---




which BW to consider?

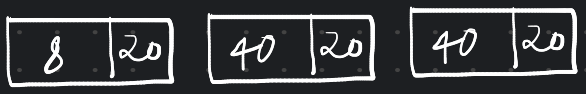
→ least. [bottleneck]

Throughput = $\eta * (1)$

---




off. = 22
MF = 1
HLF = 5
TL = 108
ID = 100




| | | | |
|---|---|---|---|
| $27+\frac{40}{8}$ | $22+\frac{40}{8}$ | 22 | offset |
| $=32$ | $=27$ | | |
| 1 | 1 | 1 | MF |
| 5 | 5 | 5 | HLF |
| 28 | 60 | 60 | TL |

---

Where is fragmentation done?
- router, not the source. (source will do segmentation)

Where are segments reassembled?
- at destination

Why not at routers?
- all fragments may not meet at a particular router
- or maybe fragmentation might be required in the future

Reassembly:

Say a datagram is fragmented into 4 fragments:




fragments may be out of order: 1, 3, 2, 4
fragments may be lost: 1, 3, 4

Reassembly at destination:

i. destination should understand that there's fragmentation:
- MF is used. But if we get the last packet, it'll have MF = 0.
Then we'll check offset.

| MF | Off | |
|---|---|---|
| 1 | 0 | - 1st fragment |
| 1 | +ve | - intermediate packet (not first, not last) |
| 0 | +ve | - last fragment |
| 0 | 0 | - no fragmentation |

ii. destination should know that all packets are from the same datagram:
- done using the identification #

iii. identifying the 1st fragment:
- offset = 0

iv. identify subsequent packets:
- we cannot do it using sorting the offsets as in case of missing fragments, we'd still combine the fragments without realizing that some are missing. we'd just calculate offsets. If a fragment has data 80B and offset 0 (i.e. the first packet), the second fragment would have an offset 10.
If 2nd fragment has data 80, offset of 3rd would be 20.
offset of next fragment = offset of current + (TL - HL*4)/8 = curroff + curr data

v. repeat it until MF = 0 (i.e. last fragment)

If any fragment is missing, all will be discarded. TCP will take care.
IP is unreliable, there are no acknowledgements in it.

Protocols and Concepts at NL:

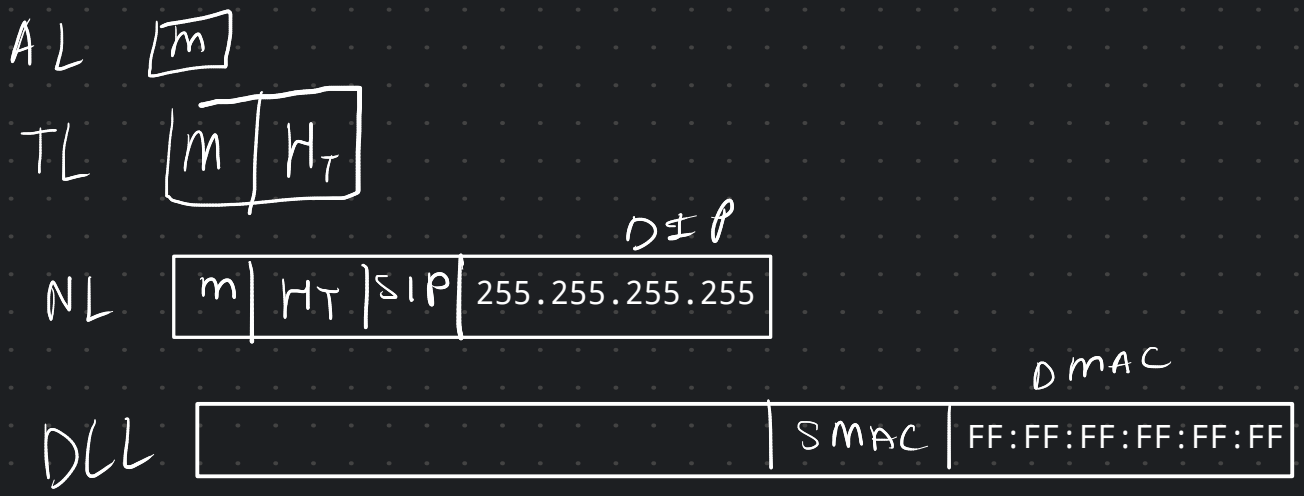


Broadcasting:

LBA:

if host wants to send a packet
to all of the hosts in a n/w,
DIP will be 255.255.255.255

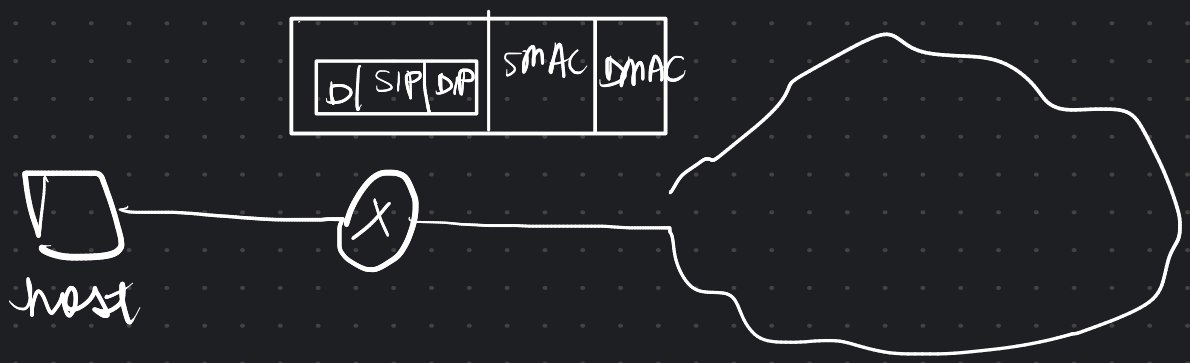
Both the LBA and DBA are converted to broadcasting in the DLL.




DBA:

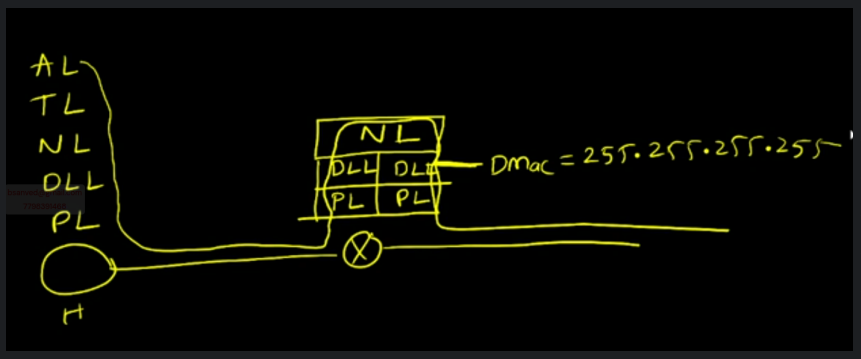
Say there's a n/w 20.0.0.0 and a host wants to send a packet to all hosts in that n/w. It'll send the packet to the router, with DIP as 20.255.255.255.
The router will put it in a frame with the source MAC and Dest Mac address (FF:FF:FF:FF:FF:FF)

Here, router will do that. In LBA, the host does that



A router will have PL, DLL, NL.
A host will have AL, TL, NL, DLL, PL

DLL and PL of routers are diff. for diff interfaces they're connected to.



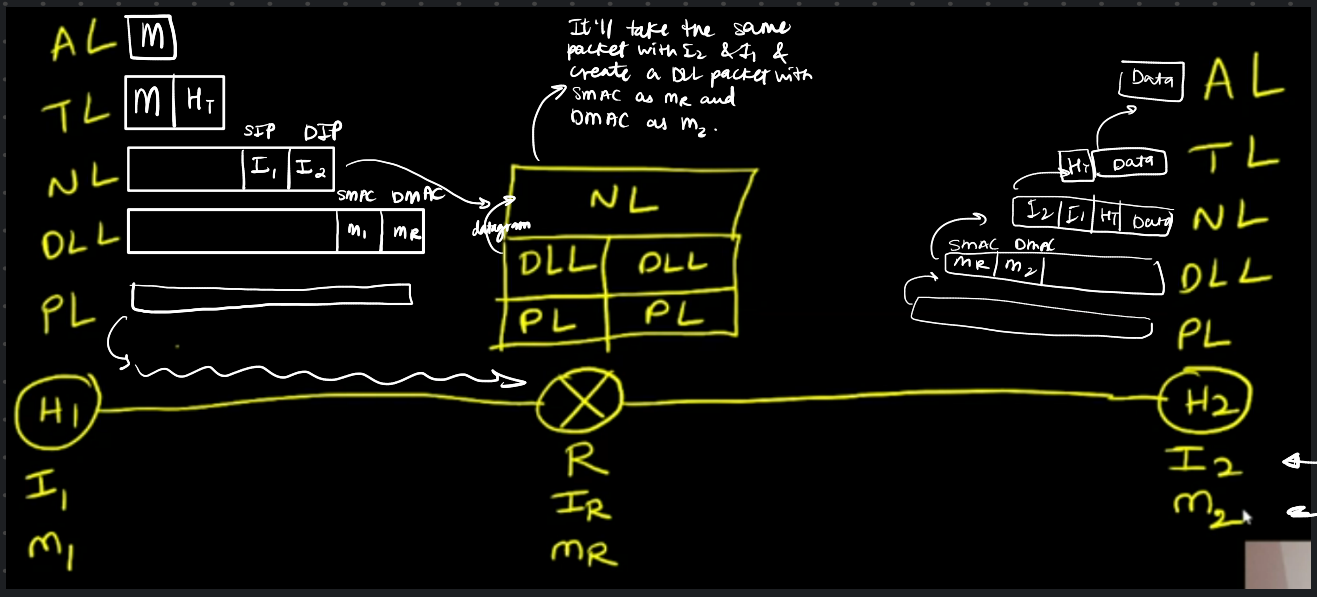

---

how many times is the NL touched?
- 4

how many times is the DLL touched?
- 6

how many times is the TL touched?
- 2





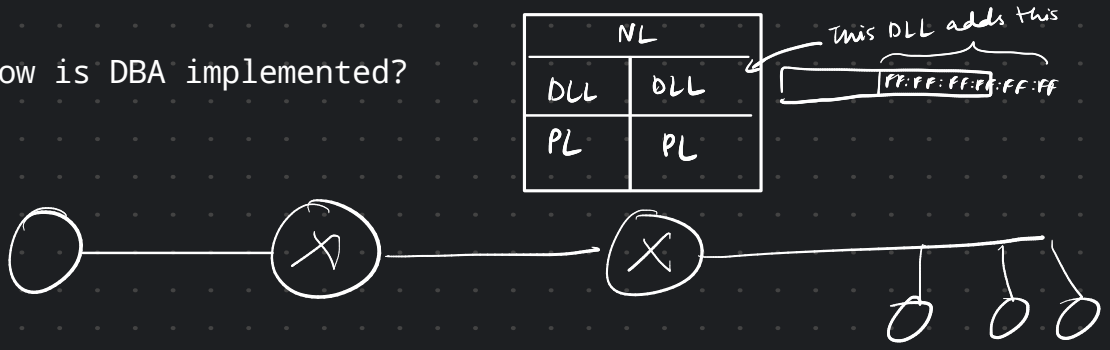

H1 knows that H2 isn't in the same n/w (using subnet mask) so it'll send packet to router (DMAC of rouer)

how will H1 know router's MAC address?
- ARP

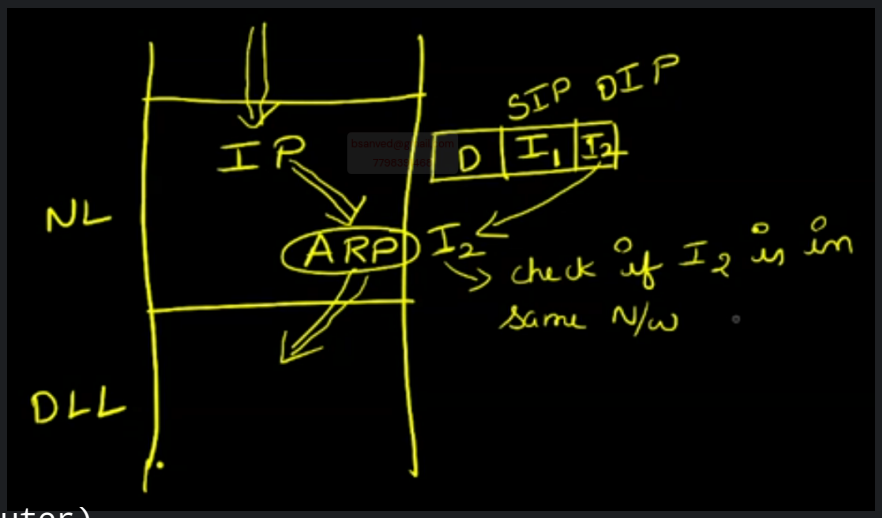
router will add DMAC as M2

how will the router know MAC of R2, i.e. M2?
- ARP

How is DBA implemented?



ARP: Address Resolution Protocol



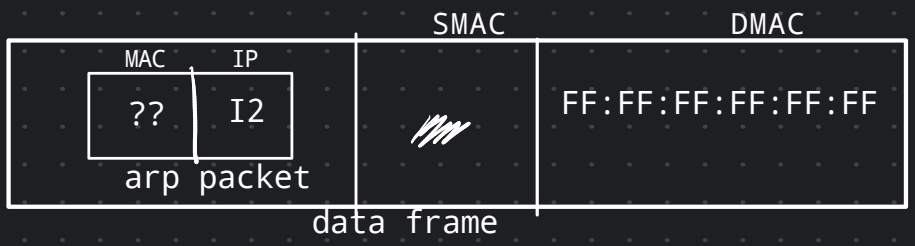

ARP works in the NL below IP.
It'll get a packet with I1 and I2 (SIP and DIP)

It'll check if I2 (dest) is in same network.
If yes, then it'll get the MAC address of dest,
else it'll get MAC addr of router.

It'll give all that to DLL.
(packet and M2, or packet and M_router)

Consider the situation:



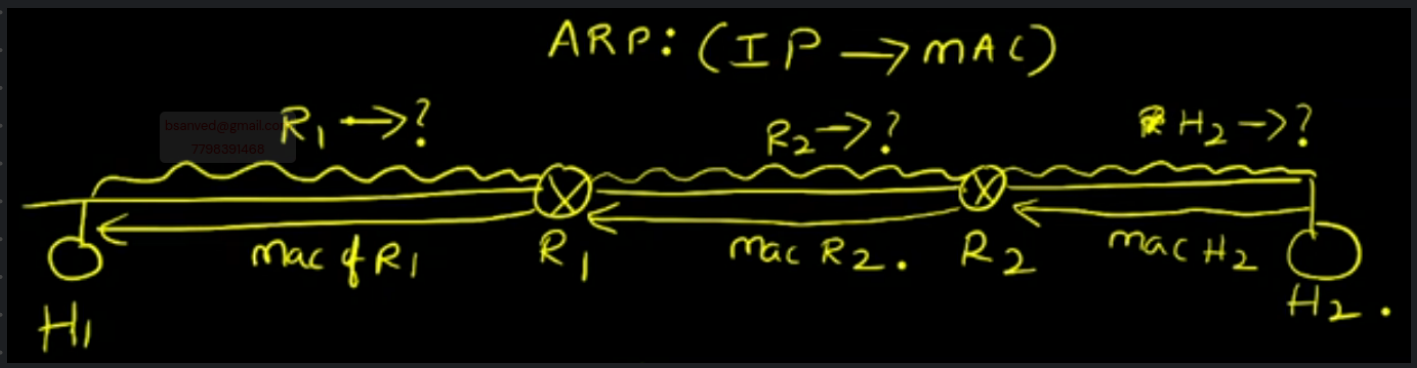




ARP packet will have DIP as I2; It'll be put in data frame and broadcast to the whole network. When I2 will see it, I2 will reply with it's MAC address.

ARP req -> broadcast
ARP reply -> unicast

btw broadcasting done at DLL is always limited to networks. The routers act as a boundary to networks.

If there are multiple routers, ARP req will be sent to find MAC addresses of subsequent routers and those in turn will send ARP reqs to the dest.



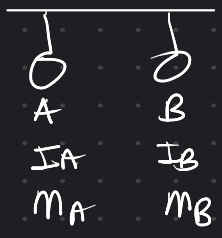

NOTE: ARP always works in the same network.

Why does ARP only work in the same network?

as broadcast at DLL
doesn't cross network boundaries




ARP Table: when A sends an ARP request and it gets back a response, it'll save details in table




| IP | MAC | validity |
|---|---|---|
| IB<br>... | MB | 10 mins |

for some time (here next 10 mins), A will save MAC address of B. It'll reuse the MAC address.

why the validity? as the MAC address may change.

why will MAC change?
too many packets come in. so there'll be wear and tear. so the NIC may have to be changed (due to overheat). even though MAC is permanent, the NIC itself could be changed.
It isn't a frequent change, but it has to be accommodated.

---

127 - special or loopback address




say we want to test the NIC. We'd have to send a message, and the NIC would send it back without putting it onto the network.

For this we use the address 127.__.__.__

any # except 127.0.0.0 and 127.255.255.255

so say the DIP is 127.0.0.1, the packet will go to the DLL and come back. It'd mean that the NIC is working

AL } provided by app^n dev.
TL } provided by OS
NL }
DLL } present on NIC
PL }

nothing, pure h/w

why is it imp?
say client sends req to server. and fails. the router could be faulty, server could be, or client could be. Better to start checking at the NIC of the client.

another application of 127: client server in the same host

say we've built an application and we only have 1 host. To send message to server in the same host, we'd put in DIP = 127.__.__.__ and DP (destination port) specified as well.
The message will go till the DLL and come back.

127.0.0.1 is generally called localhost.

RARP: Reverse ARP

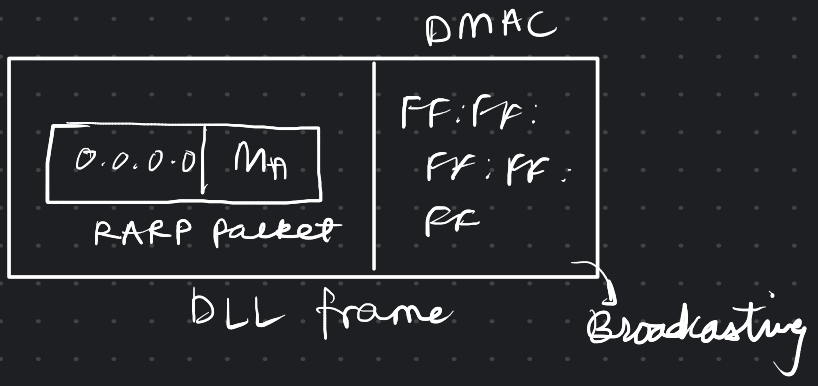
ARP is IP to MAC, RARP is MAC to IP

If you shutdown the computer, you'll lose IP and you'll have to ask for it again




say we don't have IP

we'd put IP 0.0.0.0 in RARP packet.
that'd be put into a DLL frame and then broadcast it.
In dest MAC we'll put FF:FF:FF:FF:FF:FF




Somewhere in the n/w there's the RARP server.
Essentially, we send a packet saying that we don't have an IP.
The RARP server will reply by changing 0.0.0.0 to some IP.
The RARP server has a table. (MAC, IP)

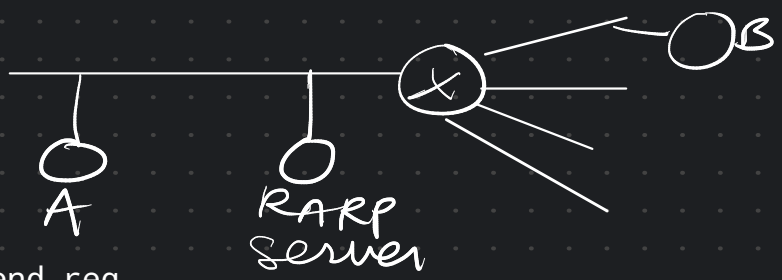
Just like ARP,
RARP request -> broadcast
RARP reply -> unicast

RARP is used to find IP addr of own host
ARP is used to find MAC addr of other host

RARP table:
if we have 100 hosts, we'll need 100 IP addresses
So the table is static, not dynamic
(that's a disadvantage)

| MAC | IP |
|---|---|
| M1 | I1 |
| M2 | I2 |
| ... | |
| M100 | I100 |

say we have 4 subnets.




A here will send RARP req
and get back response

B here wouldn't be able to send req
as req won't cross network

So every subnet should have its own RARP server

Disadvantage: many RARP servers are required.

RARP is really old and isn't used anymore.
BOOTP and DHCP are used.

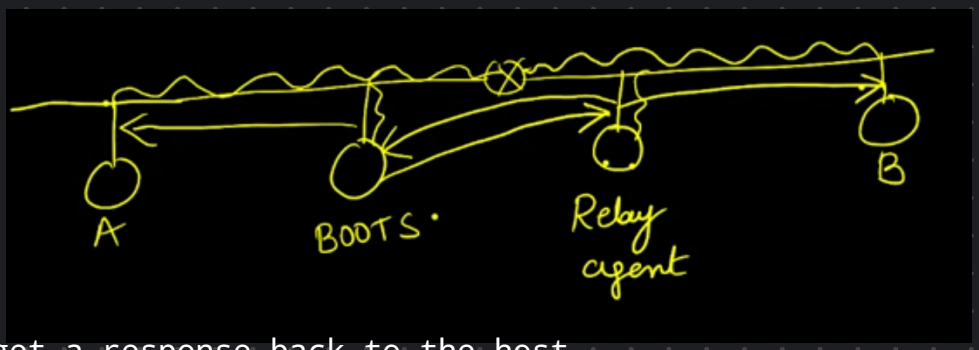
RARP works at NL.
Only protocols at AL are called servers.
So technically, RARP isn't a server.

Disadvantages of RARP:

- every network should have an RARP server

- Mapping table is static (# of IP addresses > # of hosts, if we have 100 IPs, we can only have 100 hosts)

BOOTP: Bootstrap Protocol




Hosts would send a req, which will be captured by the relay agent.
The relay agent would send it to the BOOTP server and get a response back to the host.
So # of BOOTP servers reqd = 1

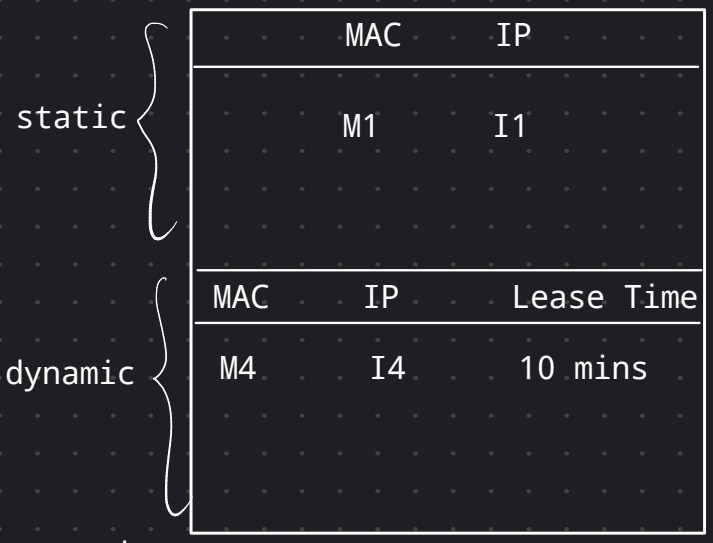
Advantage:
- In case of RARP servers, managing multiple RARP tables may lead to errors. In BOOTP, there's only 1 table.

Disadvantage:
- Mapping table is still static.

DHCP: Dynamic Host Configuration Protocol

- same as BOOTP: there'll be relay agents in diff networks

- but, DHCP table has 2 parts: static and dynamic part




only servers need to have fixed IPs

a host only needs IP for the time it uses the network

so it'll be given an IP from a pool of IP addresses for a specific duration.

after lease time, the lease must be renewed.

advantages:

- only 1 DHCP server is needed
- mapping table is dynamic

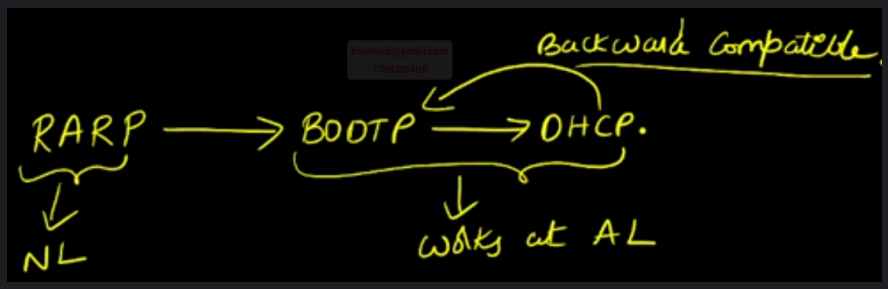
Say we have 100 users. Only 50 are online at any time. How many IPs are reqd in:

RARP: 100
BOOTP: 100
DHCP: 50

## BOOTP and DHCP work at AL, RARP works at NL

DHCP is backward compatible

if we replace BOOTP server with DHCP server, everything still works.
BOOTP clients can contact DHCP servers.
You don't have to upgrade s/w




so if we have a network of BOOTP clients, to upgrade to DHCP, we don't have to upgrade the clients. Only upgrade the server to DHCP server.

# ICMP: Internet Control Message Protocol

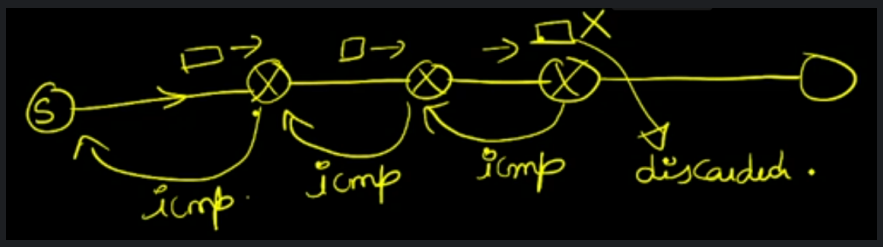
ICMP is above IP in the NL.
So ICMP packet travels in IP datagram.




## Types of ICMP

| Error Handling or Feedback Messaging | Request and Reply |
|---|---|
| - TTL exceed | - echo request and reply |
| - parameter problem | - time stamp request and reply |
| - source quench | - network mask req and reply |
| - source redirect | - router solicitation and advertisement |
| - destination unreachable | |
| - dest host unreachable | |
| - dest port unreachable | |

ICMP gives feedback if any IP is lost



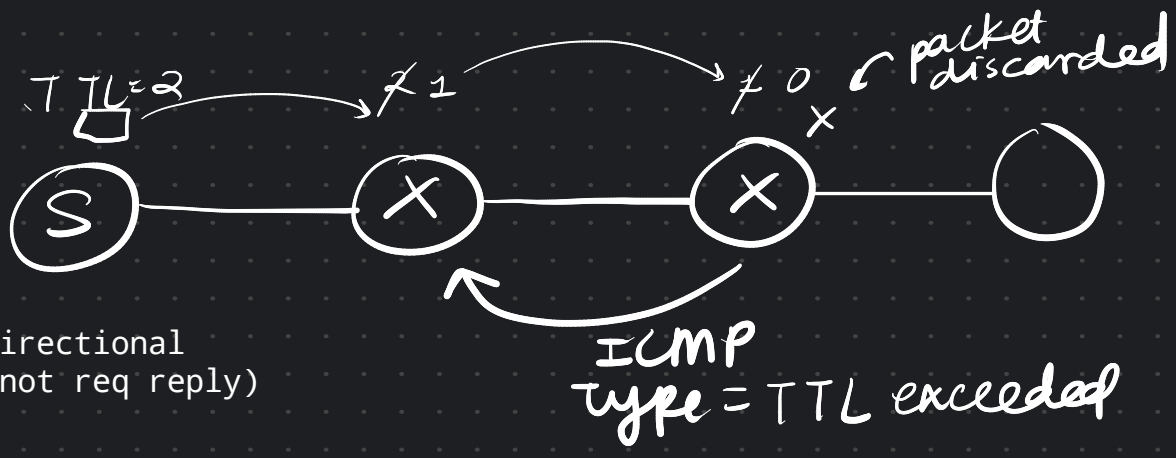

It looks like ICMP is adding reliability to IP,
but that is NOT true.

If a packet is discarded at some point, then ICMP is generated. But if ICMP is discarded, then no ICMP is generated.

IP + ICMP is still a bit unreliable.

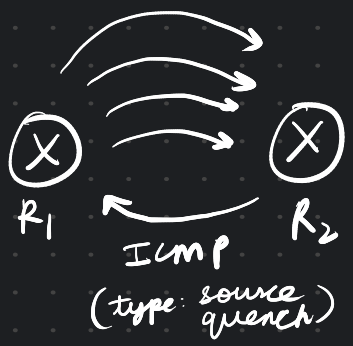
What if ICMP is lost?
TL protocols will handle packet loss.

- TTL Exceed:



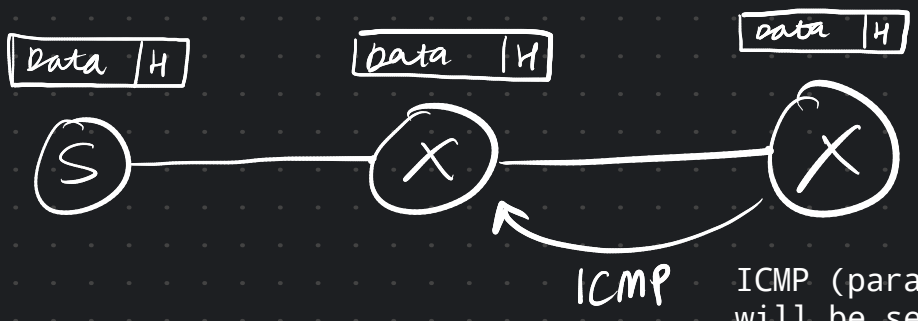

It is one directional
(feedback, not req reply)

- Source Quench




If R1 sends too many packets
to R2,
R2 will send ICMP (source quench)
to ask R1 to stop.

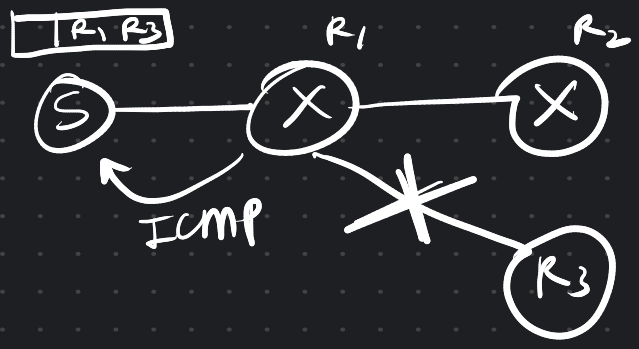
- Parameter Problem:




if there's a
problem with
setting of
fields in header
then packet
will be disc-
arded.

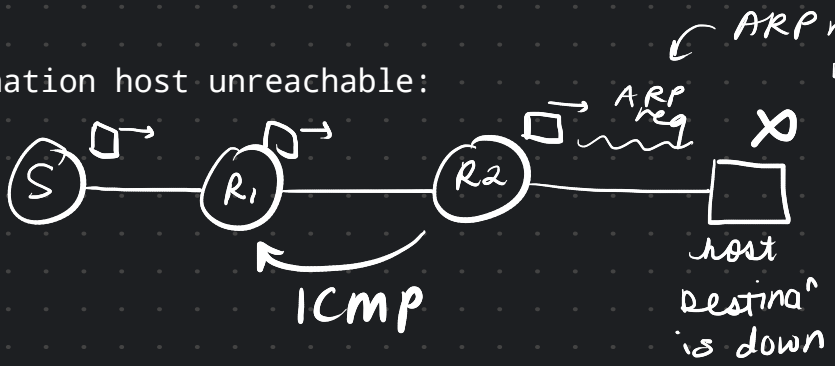
ICMP (parameter problem)
will be sent

e.g. say packet has strict source routing

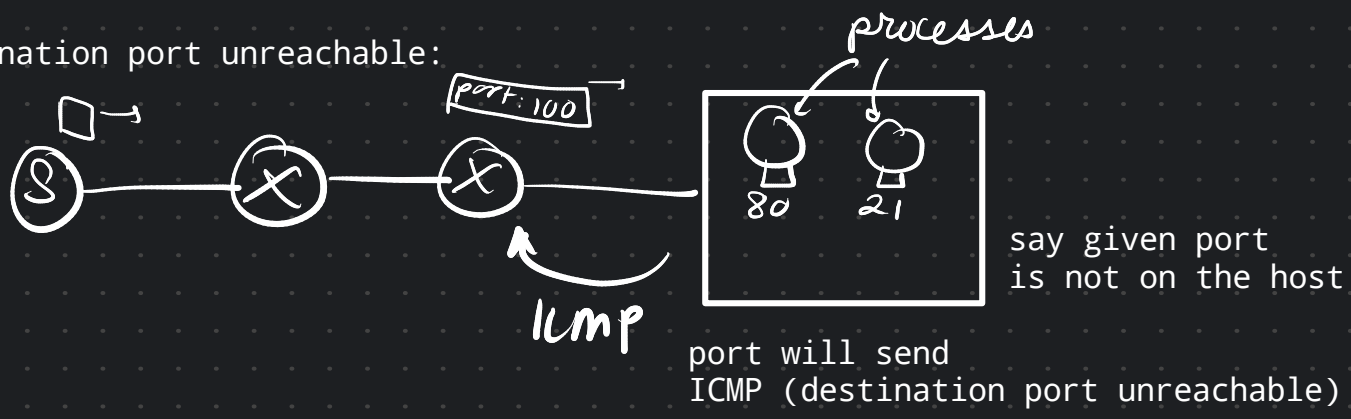



say route specified is R1 R3
but when R1 gets packet,
R3 is down.
R1 will discard a packet,
and send ICMP (parameter problem)

- destination host unreachable:

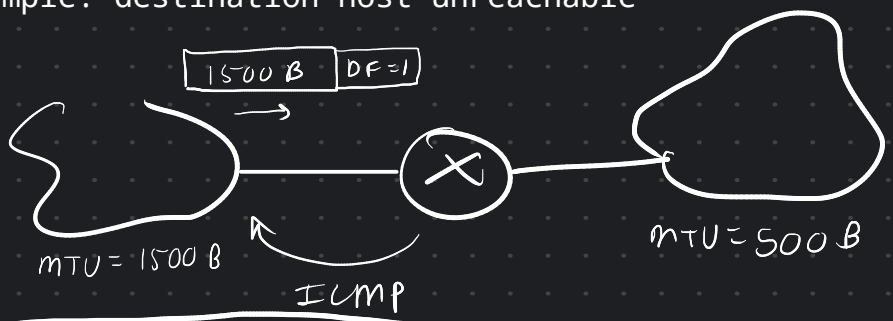



- destination port unreachable:



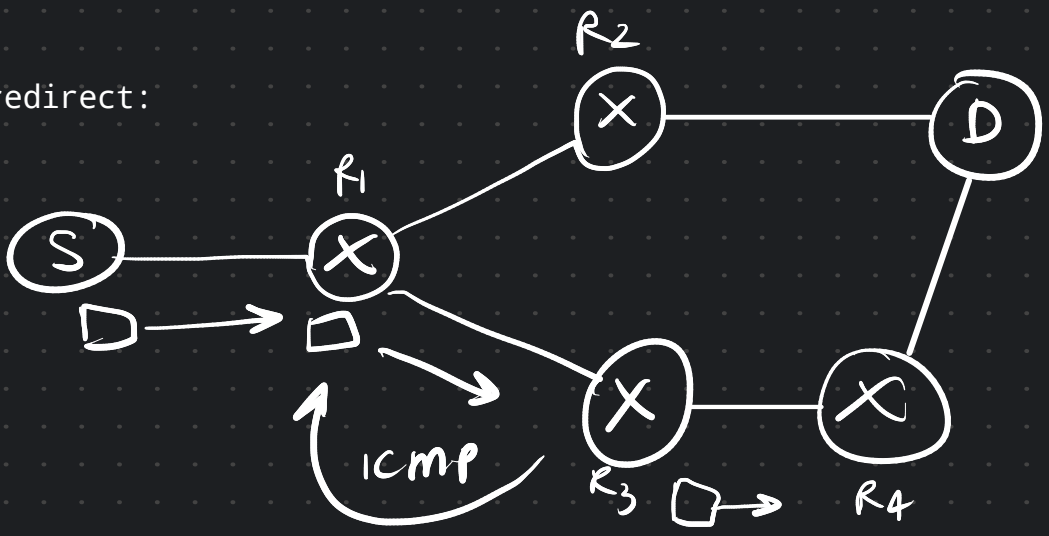

this is the only case where the receiver itself will send back ICMP

example: destination host unreachable




say DF=1, but size exceeds the MTU on receiver
router will send
ICMP (destination host unreachable, fragmentation reqd, DF=1, MTU=500B)

- source redirect:




here R3 sends back ICMP (source redirect) saying hat R2 would be the shorter path.
the router would suggest better route.
(ONLY IF THE ROUTER KNOWS)

packet inititally sent to R3 will still be forwarded, not discarded, but alongside, an ICMP packet is sent. (for next packets)

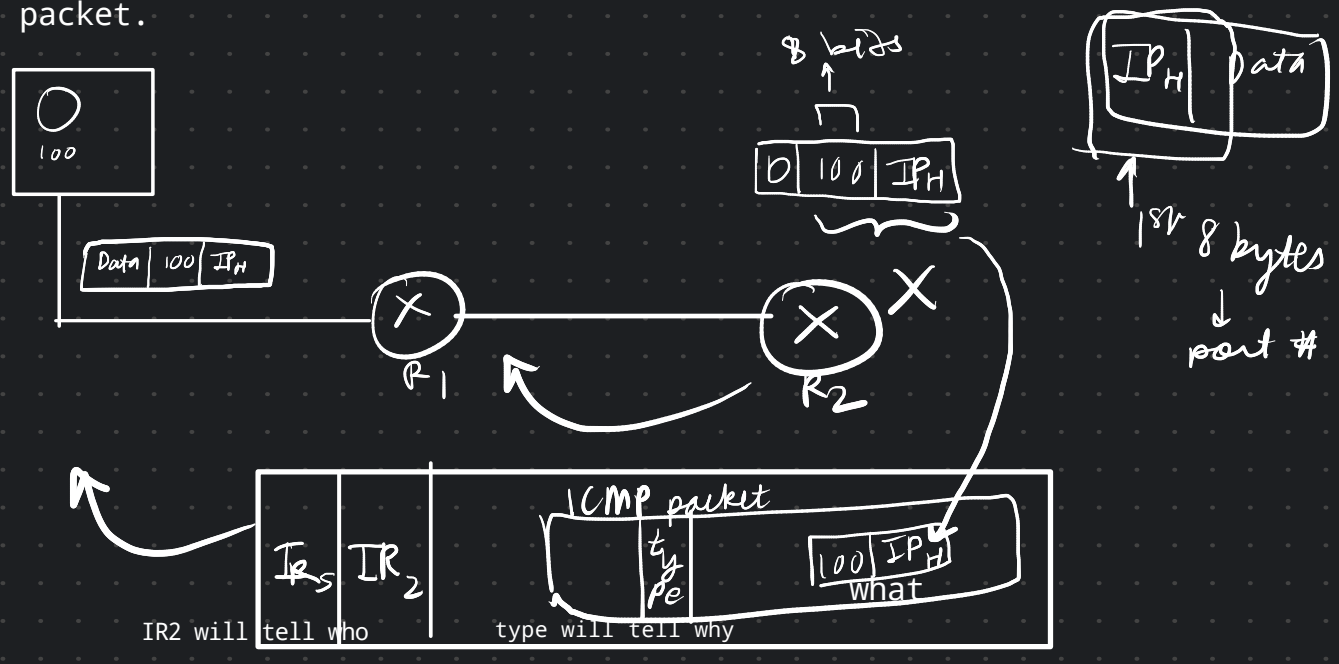
Till now we saw all discarded packets. Here ICMP is sent, but packet isn't discarded.

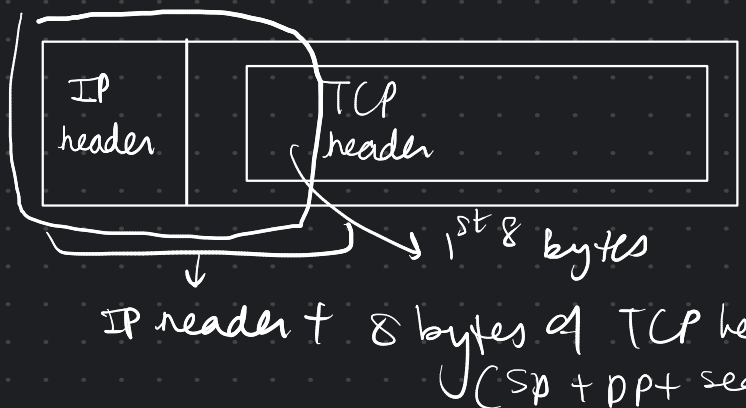
---

when a packet is discarded, the sender needs to know:
- what packet is discarded
- why is it discarded
- who discarded it

what, why, and who

assume that there's a host and a process with port # 100. It'll send a packet.





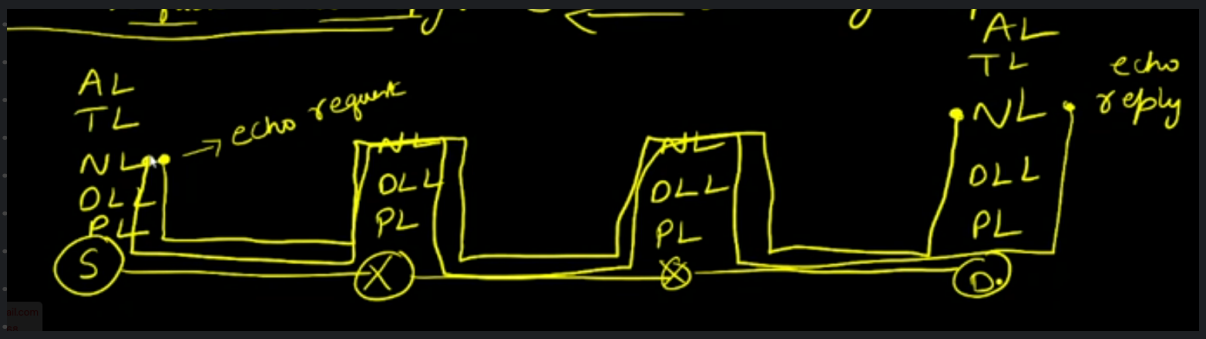

this part will tell what packet is discarded

In general, ICMP messages originate from the NL

- echo request and reply
- time stamp request and reply
- network mask req and reply
- router solicitation and advertisement

- echo request and reply

any req and reply is 2-way (2-way ICMP)




It is used to test if all the network layers are working or not. (NLs of source, routers, and dest)

PING (packet internet groper): to check if a host is available, i.e. if its NL is working

many servers disable echo reply. (Because it is misused)
Ping attack: source sends too many ping packets, the destination goes down due to heavy load. So protection needs to be implemented.

- router solicitation:

Say a host wants an internet connection. It needs
- IP address (using DHCP or BOOTP)
- subnet mask (using ICMP)
- default router/ gateway IP addr. (ICMP)
- DNS server IP addr.

To get default router/gateway IP:

when we have a network connected to a router, there'll be a broadcast asking for default router IP
the router will send ICMP saying I am the default router
ICMP (router solicitation)

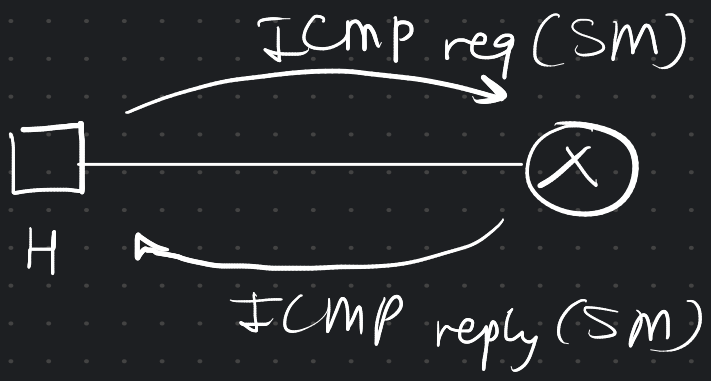
once it gets the default gateway, then the DNS server will be asked for.

- router advertisement:

  say a network is connected to some routers.
  when a new router is added, it'll send an advertisement saying that it is available, if anyone wants to then they can use it.

- network mask request and reply:

for a new n/w:
- get IP
- get def router IP
- ask DR for SM



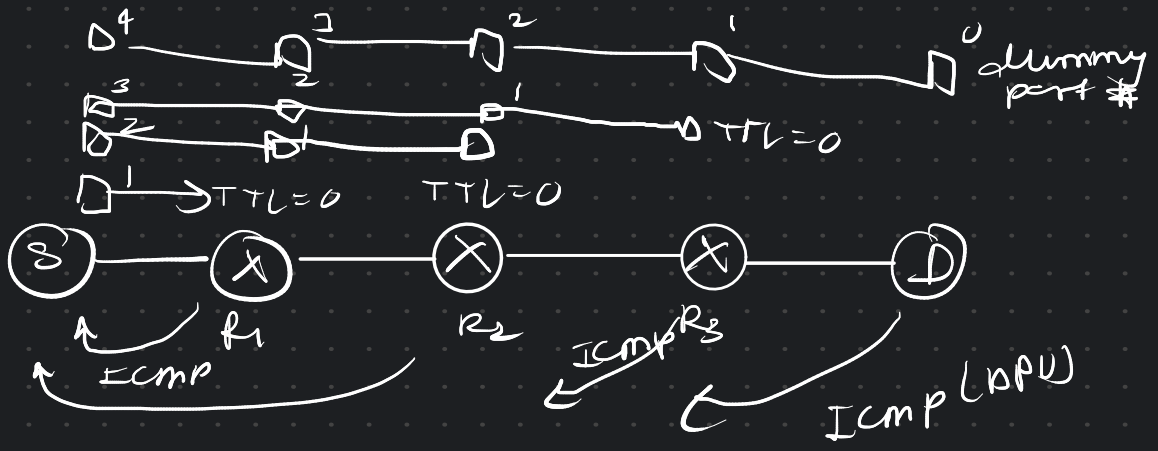

- timestamp req and reply:

  for 2 hosts in diff time zones, they have to be synchronized.
  timestamp ICMP is used to synchronize clocks.

  but it isn't used anymore. (really old)

there are many applications of ICMP. 2 popular ones are:
- traceroute
- PMTUD (Path MTU Discovery)

Traceroute:



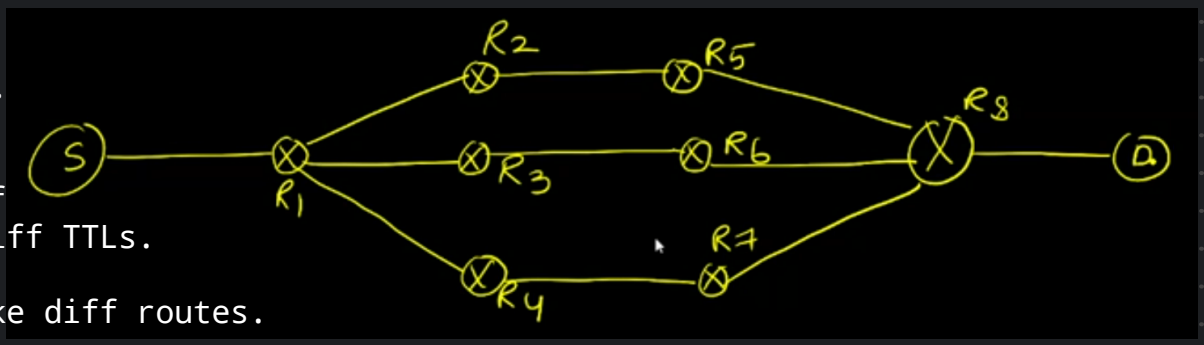

we want to make R1, R2, R3, and D all send ICMP packets.

one simple way:
send packets will TTL = 1, 2, 3, 4 respectively
R1, R2, R3 will send ICMP (TTL exceed)
But for packet with TTL = 4, at dest TTL becomes 0, but it won't produce ICMP as it is the destination and the packet is kept
So we'll send packet with dummy port #
Dest will send ICMP (dest port unreachable)

so we'll know that the whole route is S R1 R2 R3 D

problem:

consider a case like this.

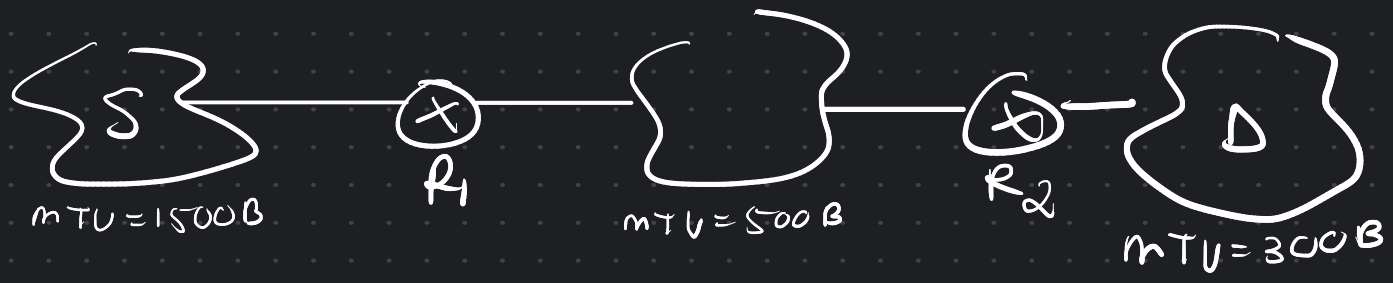



We'll send diff packets with diff TTLs. Those diff packets may take diff routes.

So exact path is not guaranteed by traceroute.

maybe we'd get ICMPs something like R1 R2 R6 R8

Path MTU Discovery:




best size that the source can send to destination is 300B to avoid fragmentation.

how will source discover the path MTU?

for the entire path, the path MTU is the minimum MTU.

source will send 1500 B packet, DF=1
R1 will send back ICMP (mtu=500, DF=1, DHU)
again, the source will send 500 B packet, DF=1
R2 will send back ICMP (mtu=300, DF=1, DHU)
now source will send 300 B packet

but port # will be dummy, so we'll get ICMP (DPU) from dest

so we'll get path R1 R2 D and know that path MTU is 300B

Switching: putting an incoming packet onto one of the available outgoing links
Routing: creating routing tables
Flooding: putting an incoming packet onto all available outgoing links

Switching uses routing tables if they're available. Otherwise it uses flooding.

Advantages of flooding:
- no routing: usually routing algos are complex.
- if a packet is sent from source to dest, it'll be sent on all possible paths. so the shortest path is guaranteed.
- highly reliable: multiple copies of a packet are sent. (only 1 is kept)

Disadvantages of routing:
- costly (implementation of algorithms)
- shortest path isn't always guaranteed
- not very reliable

Disadvantages of flooding:
- duplicate packets are received
- traffic is very high

Advantages of routing:
- no duplicate packets
- low traffic

## Types of Routing Algorithms

### Static

disadvantages:
- routing table doesn't change automatically with changes in traffic and topology

### Dynamic

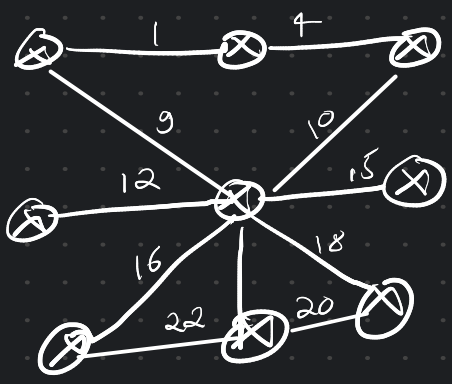
#### DVR

routing tables change automatically with changes in traffic and topology

#### LSR

consider a network:
if we manually compute routing tables for each router, it is static routing.

Problem:
no one knows the graph of the entire internet. Everyday, new routers are added and removed.
So this isn't possible for the internet, but only within organizations.



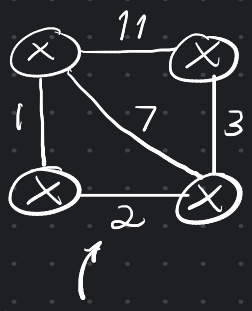

---

DVR: Distance Vector Routing

consider a n/w.
the routing table would look like:

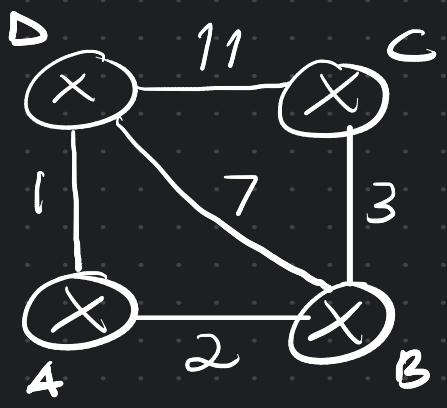
| dest | dist | next hop |
|---|---|---|
| A | 10 | B |

(to go to A, distance is 10, route is via B)




these #s are weights.
they may be propogation delay, queueing delay, distance, cost, or anything else

step 1: all routers will create routing tables using knowledge about their neighbors. Only direct links are used.




At A:

| dest | dist | next hop |
|---|---|---|
| A | 0 | A |
| B | 2 | B |
| C | inf | - |
| D | 1 | D |

in next hop we essentially say that we have to send packet to that node to reach dest

At B:

| dest | dist | next hop |
|---|---|---|
| A | 2 | A |
| B | 0 | B |
| C | 3 | C |
| D | 7 | D |

at C

| dest | dist | next hop |
|---|---|---|
| A | inf | - |
| B | 3 | B |
| C | 0 | C |
| D | 11 | D |

at D

| dest | dist | next hop |
|---|---|---|
| A | 1 | A |
| B | 7 | B |
| C | 11 | C |
| D | 0 | D |

step 2: exchange distance vectors with neighbors

At A, distance vectors will be received from B and D. lly at B from A, D, and C. At C, from B and D. At D, from A, B, and C.

At A:

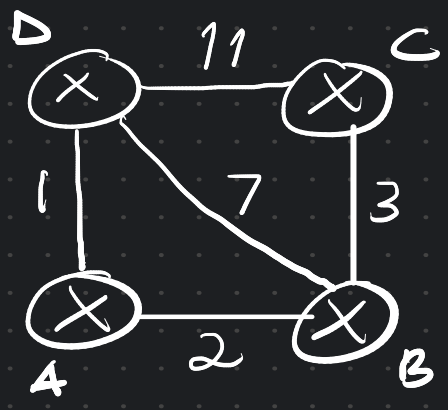
| From B | From D |
|---|---|
| 2 | 1 |
| 0 | 7 |
| 3 | 11 |
| 7 | 0 |

old RT at A

| dest | dist | NH |
|---|---|---|
| A | 0 | A |
| B | 2 | B |
| C | ∞ | – |
| D | 1 | D |

new RT at A

| dest | dist | NH |
|---|---|---|
| A | 0 | A |
| B | 2 | B |
| C | 5 | B |
| D | 1 | D |




$dist(AB) = \min \begin{cases} A \to B + B \rightsquigarrow B \\ A \to D + D \rightsquigarrow B \end{cases}$

To reach B from A, we go to either of 2 neighbors. B or C.

↳ direct edges

$= \min \begin{cases} 2 + 0 = 2 \\ 1 + 7 \end{cases}$

↑ we got on exchanging routing tables.

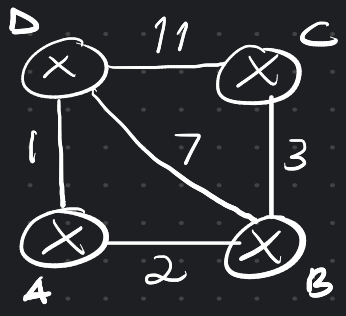
$AC = \min \begin{cases} A \to B \quad B \rightsquigarrow C \\ A \to D \quad D \rightsquigarrow C \end{cases}$

$= \min \begin{cases} 2 + 3 = 5 \\ 1 + 11 \end{cases}$

$AD = \min \begin{cases} 1 + 0 = 1 \\ 2 + 7 \end{cases}$

btw this is the Bellman-Ford algorithm

Shortcut:




At A:

From B we got

| |
|---|
| 2 |
| 0 |
| 3 |
| 7 |

$A_B = 2$

From D

| |
|---|
| 1 |
| 7 |
| 11 |
| 0 |

$A_D = 1$

To create new table,

| Dest | Dist | Next |
|---|---|---|
| A | 0 | 1 |
| B | 2 | B |
| C | 5 | B |
| D | 1 | D |

To fill 2nd row,

we add [0 7] and [2 1]

we get [2 8] put min 2, from B

3 11

2 1

5 12

7 0

2 1

9 1

At B: From A: 0, 2, ∞, 1 → BA = 2

C: ∞, 3, 0, 11 → BC = 3

D: 1, 7, 11, 0 → BD = 7

| dest | dist | next hop |
|---|---|---|
| A | 2 | A |
| B | 0 | B |
| C | 3 | C |
| D | 3 | A |

always do this first

(i 0 i)

| | A | C | D |
|---|---|---|---|
| A: | 2 | 3 | 7 |
| | 0 | ∞ | 1 |
| | 2 | ∞ | 8 |

C: ∞ 0 11 / 2 3 7 → 3

D: 1 11 0 / 2 3 7 → 3 14 7

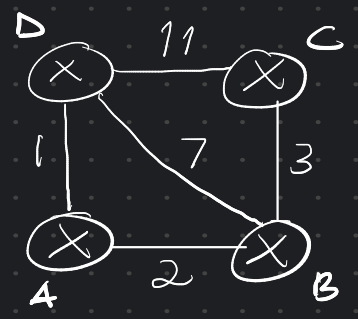
---

At C: From B: 2, 0, 3, 7 → CB = 3

D: 1, 7, 11, 0 → CD = 11

| dest | dist | next hop |
|---|---|---|
| A | 5 | B |
| B | 3 | B |
| C | 0 | C |
| D | 10 | B |

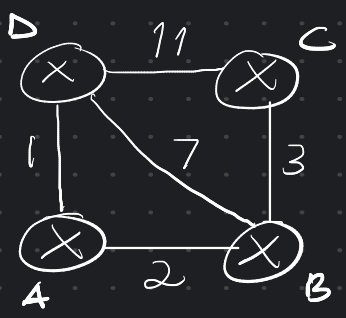
← i 0 i




we'll get final tables when all routers share info betn themselves

summary:
step 1: routers will create RT using knowledge about neighbors
step 2: distance vectors are exchanged and new routing tables are created
step 2 is repeated forever

in the exam, we won't do all this
we can tell the final routing tables directly




At A:

| Dest | Dist | Next hop |
|---|---|---|
| A | 0 | A |
| B | 2 | B |
| C | 5 | B |
| D | 1 | D |

At B

| Dest | Dist | Next |
|---|---|---|
| A | 2 | A |
| B | 0 | B |
| C | 3 | C |
| D | 3 | A |

At C:

| Dest | Dist | Next |
|---|---|---|
| A | 5 | B |
| B | 3 | B |
| C | 0 | C |
| D | 6 | B |

At D:

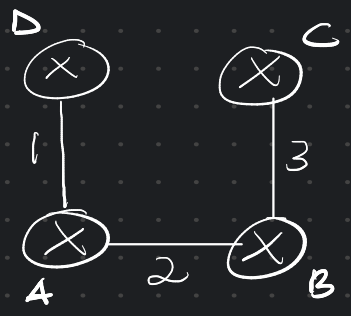
| Dest | Dist | NH |
|---|---|---|
| A | 1 | A |
| B | 3 | A |
| C | 6 | A |
| D | 0 | D |

---

Q. At A, what are the edges used? AB or AD
Q. At B? AB or BC
Q. At C? only CB
Q. At D? only DA

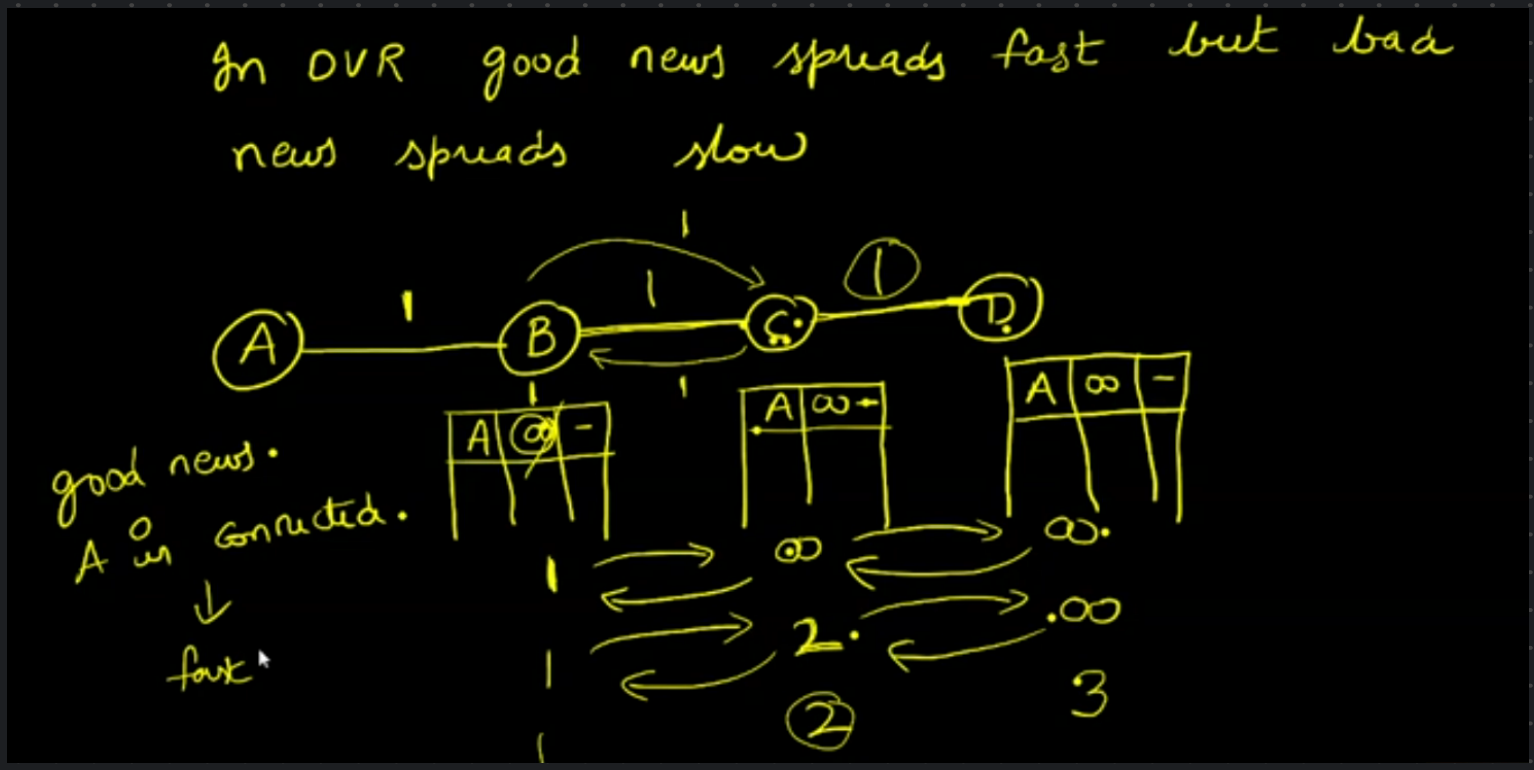
So overall only 3 edges are used: AB, BC, AD
2 more are redundant: BD, CD

The network could be reduced to:

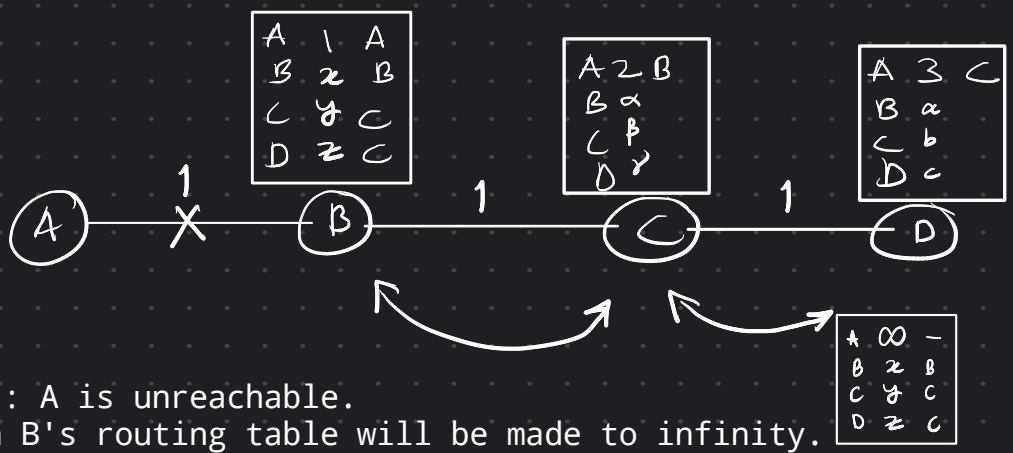



Questions:
1. Given DVs, RT?
2. Final RTs?
3. Redundant edges?

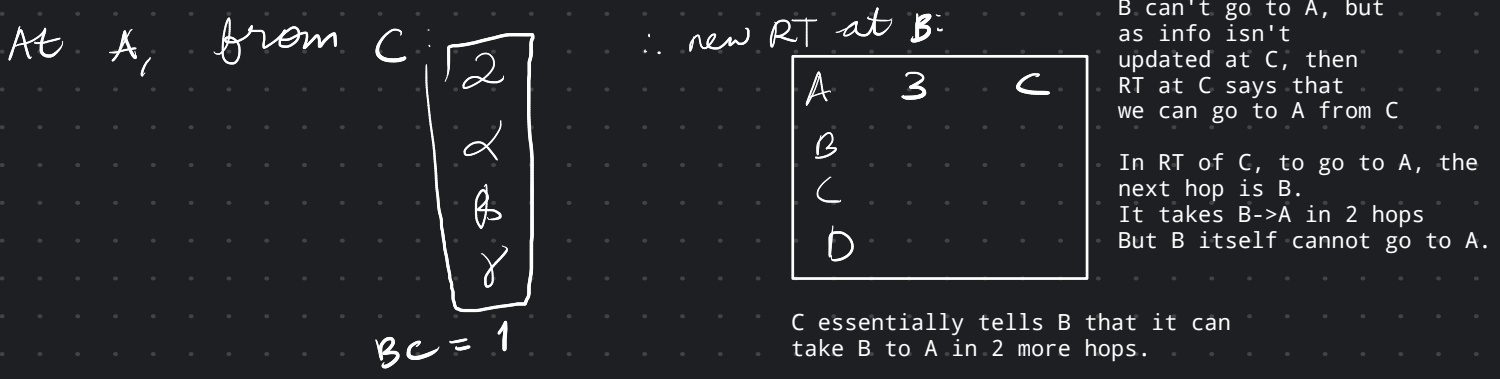
In DVR, good news spreads fast, but bad news spreads slowly.



# bad news spreads slow



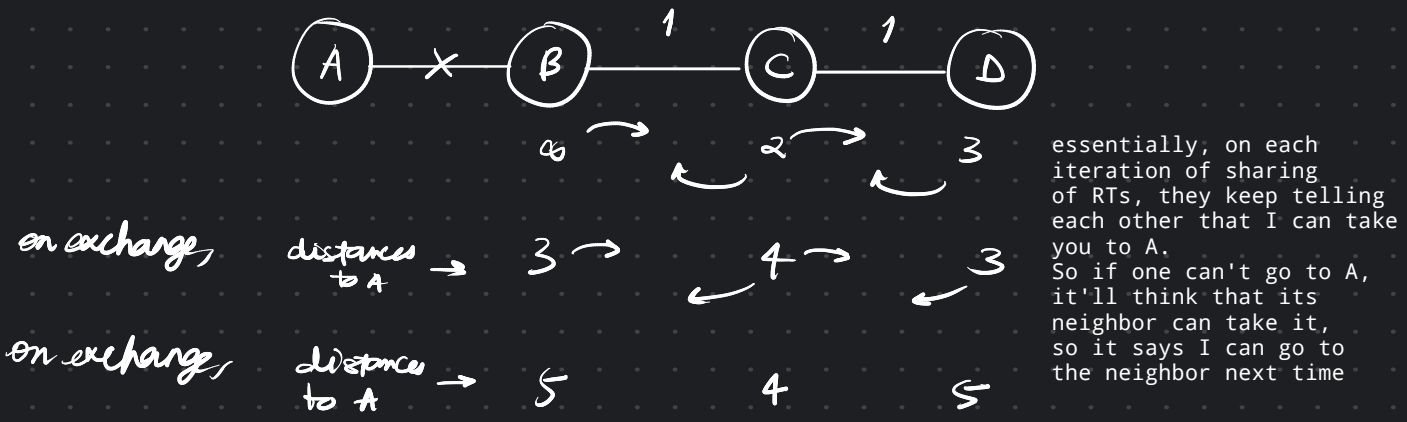

bad news: A is unreachable.
Entry in B's routing table will be made to infinity.
Then routing tables will be exchanged betn B and C, C and D




B can't go to A, but as info isn't updated at C, then RT at C says that we can go to A from C

In RT of C, to go to A, the next hop is B.
It takes B->A in 2 hops
But B itself cannot go to A.

C essentially tells B that it can take B to A in 2 more hops.




essentially, on each iteration of sharing of RTs, they keep telling each other that I can take you to A.
So if one can't go to A, it'll think that its neighbor can take it, so it says I can go to the neighbor next time

this would go on infinitely...

this is called count to infinity

and it takes a looot of time

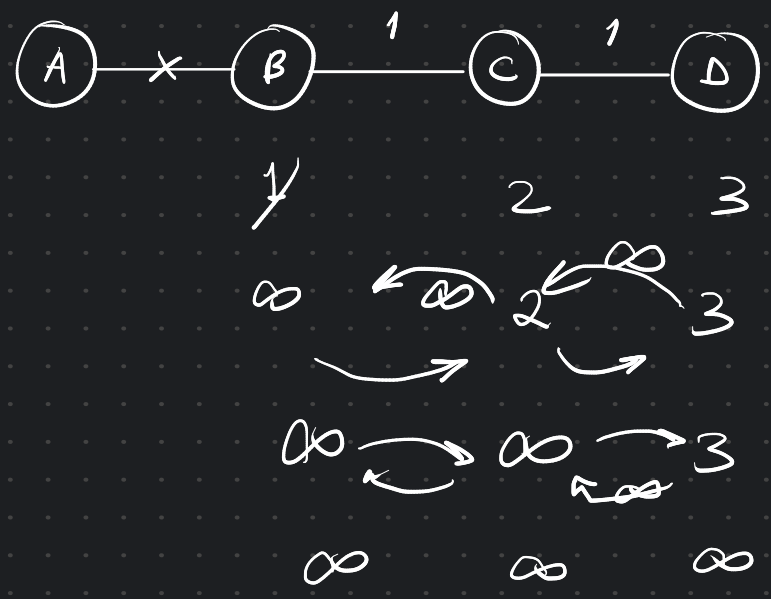
How to solve count to infinity: split horizon

problem with dvr is that only the distance is sent, not the next hop.

In the above problem, on exchanging RTs, C tells B that it has a path to A. But it doesn't tell B that the path is via B itself. B thinks that there's some other path to A via C.

B is helping C to reach A.
Next time C is helping B to reach A.

The rule is, don't help B if B is helping you.



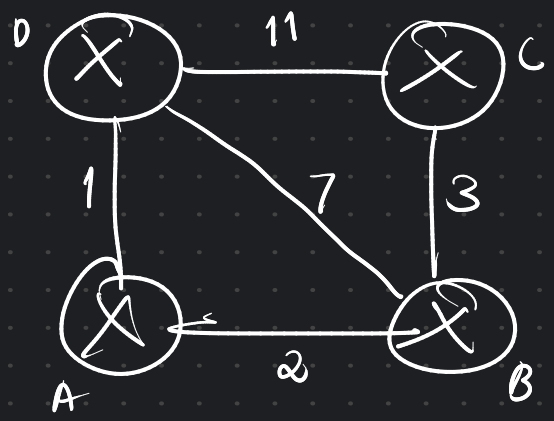

As C is depending on B to go to A, C will send infinity to B.
C can send 2 to D. D will send infinity to C.

if a node is dependent on some other node (next hop), then it'll send infinity to the other node.

Example for DVR with split horizon

RT of B:

| | | |
|---|---|---|
| A | 2 | A |
| B | 0 | B |
| C | 3 | C |
| D | 3 | A |

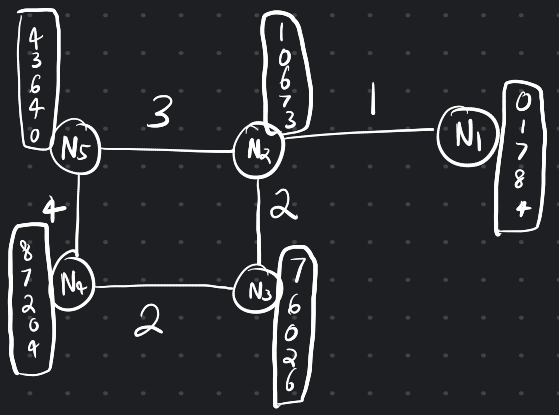



∴ B will send [∞, 0, 3, ∞] to A.

[2, 0, ∞, 3] to C

[2, 0, 3, 3] to D.

---




Q. new RT at N3 after getting distance vectors from N2 and N4?

→ this is DVR without split horizon

[if nothing given, do NOT go with split horizon]

From $N_2$ [1, 0, 6, 7, 3]

From $N_4$ [8, 7, 2, 0, 4]

$N_3$ $N_2$ 2

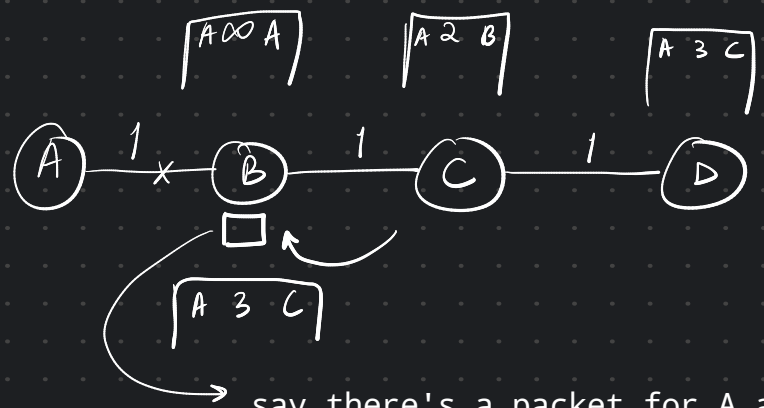
$N_3$ $N_4$ 2

| | | |
|---|---|---|
| $N_1$ | 3 | $N_2$ |
| $N_2$ | 2 | $N_2$ |
| $N_3$ | 0 | $N_3$ |
| $N_4$ | 2 | $N_4$ |
| $N_5$ | 5 | $N_2$ |

Disadvantages of DVR:

- count to infinity
- slow convergence (final routing tables take time)
- loops are possible

loops?

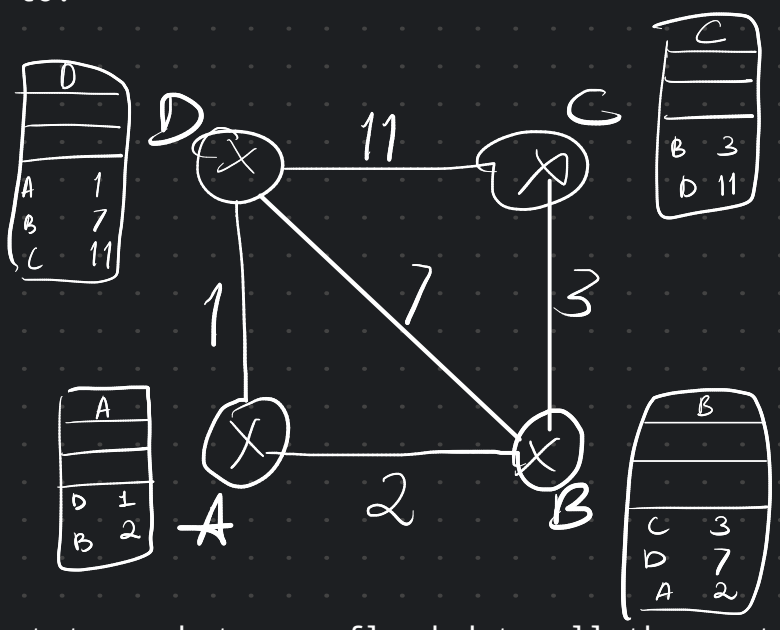



say there's a packet for A at B. Next hop is C.
So B will forward it to C.
At C, next hop to A is B. C will forward it to B.
B will again forward it to C.
And so on...

DVR is really old (1980's). LSR is old (1990's).

LSR: Link State Routing

Step 1: Every router will create link state packets about links it is connected to.




Step 2: All link state packets are flooded to all the routers, not just the neighbors.

key difference betn DVR and LSR. (not just neighbors)
so DVR is based on local, while LSR is based on global knowledge

Step 3: All routers construct the graph in memory using the LSP's.

At A: from B

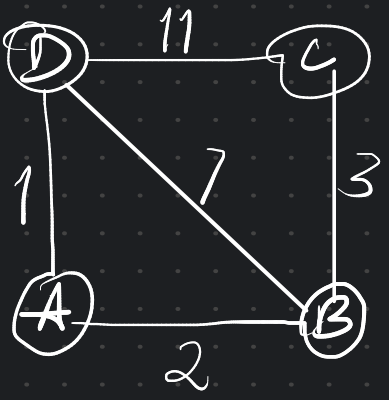
| | |
|---|---|
| A | 2 |
| D | 7 |
| C | 3 |

from D

| | |
|---|---|
| A | 1 |
| B | 7 |
| C | 11 |

from C

| | |
|---|---|
| B | 3 |
| D | 11 |




Now A will apply Dijkstra's algorithm for shortest path to find routing table of A

Step 4: The nodes will use Dijkstra's to find the routing tables.

DVR grows step by step. 1 edge -> 2 edges -> 3 edges -> ...

LSR is faster.
LSR converges much faster.

But LSR requires more BW, as the link state packet size is greater
and it floods packets

Problem with LSR:
- flooding: creates duplicate packets.
  so sequence #s are required.
  (Btw identification #s are for IP datagrams, here we use seq #s. potayto, potahto)

all link state packets have a seq # field

Routers store "latest sequence #s"

At A:

| | seq # |
|---|---|
| B | 100 |
| C | 200 |
| D | 300 |

Say we get packet from B with seq # 50.
We already have a packet 100 though.
So the 50 one is an old packet.
So A will discard it.

Problem:
If due to some error, the seq # gets corrupted, say at B instead of 100, 1024 seq # has come. Then all from 101, 102, ..., 1023 are discarded.
So a lot of information is lost.

So sequence #s aren't used for a long time.
Say after 10 ms, we'll ignore the seq # and just accept whatever packet comes.



| DVR | LSR |
| --- | --- |
| - 1980's | - 1990's |
| - BW requirement is less as DVs are small and there's no flooding | - BW req is high |
| - based on local knowledge (neighbors) | - based on global knowledge (entire n/w) |
| - traffic is less | - traffic is high |
| - periodic updates are done | - periodic updates are done |
| - converges slowly | - converges faster |
| - count to infinity problem | - no count to infinity |
| - persistent looping | - transient looping |
| - RIP uses DVR | - OSPF uses LSR |
| - simple | - complex |

RIP: Routing Information Protocol

DVR is an algo. RIP is an implementation of the algo.

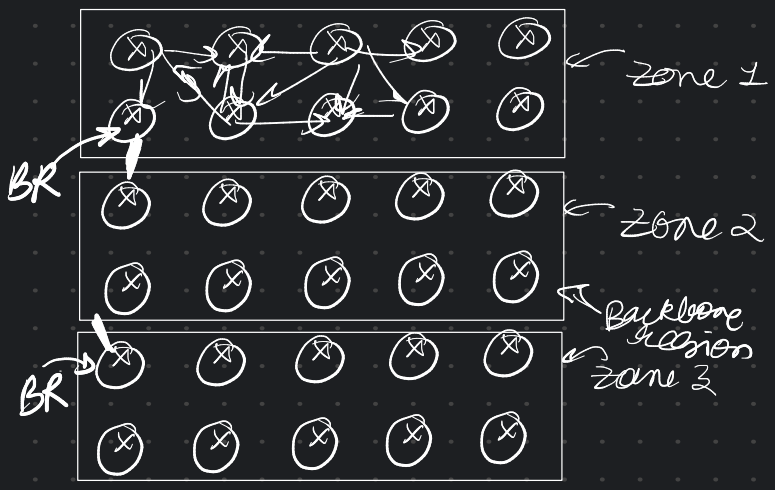
In RIP, 16 is taken as infinity.

Every edge has weight 1.

---

OSPF: Open Shortest Path First

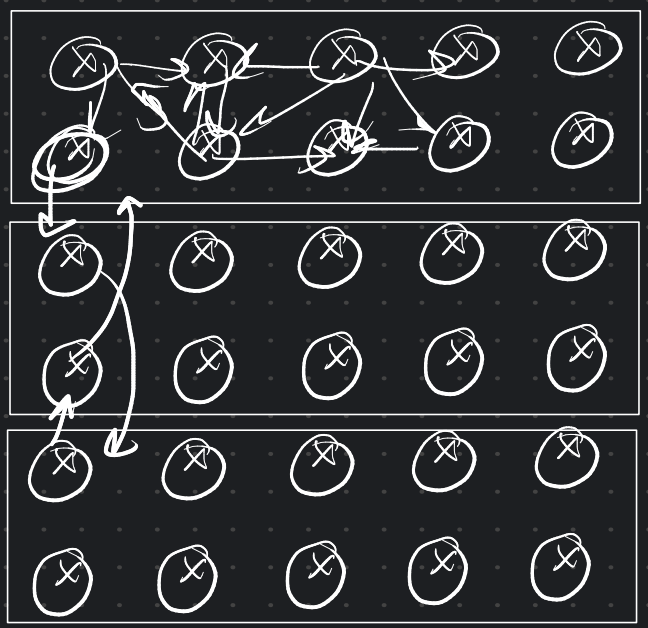
All the routers are divided into several regions/zones.




- Flooding is only done within a zone.

- Every zone will have a border router. Border routers will be connected to backbone region.
So border routers will transmit data to backbone region

LSR uses flooding, so it is high traffic.
So OSPF uses regions and flooding is done within those regions.



flooding is restricted

The edge weights on the network are determined by the network admin.
In RIP, 1 is used for weights.

---

EIGRP: Enhanced Interior Gateway Routing Protocol

- a combination of RIP and OSPF

---